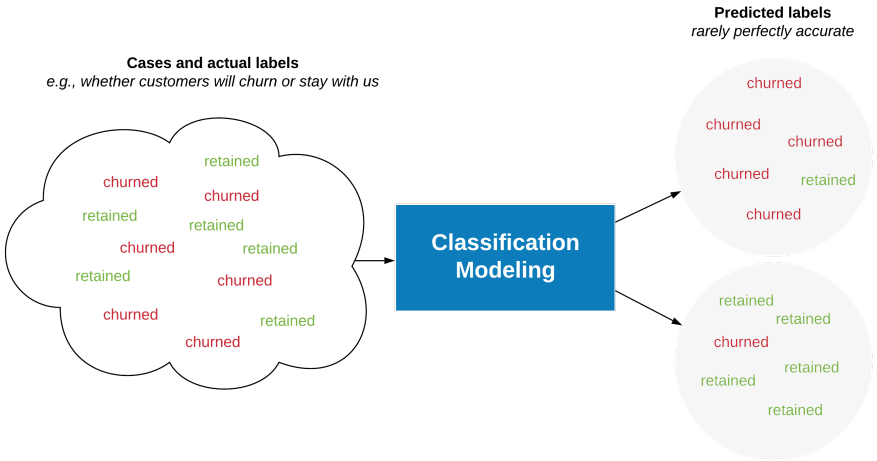
Chapter 13: Classification Modeling

13.1Introduction

Modeling categorical data differs from modeling numeric data because categorical values do not have an inherent numeric ordering. This does not mean that categories never have a meaningful sequence in the real world, but rather that classification models deliberately ignore any assumed order. Instead of fitting a regression line to ordered values, classification modeling estimates the likelihood that each observation belongs to each possible category. We refer to this process as classification modeling Assigning items in a collection to predefined target categories or classes with the goal of accurately predicting the class membership for each case in the data..



For example, classifying customer behavior, product categories, or loan approval outcomes involves learning how strongly different attributes contribute to category membership. One category is not greater than or less than another, so numeric regression techniques that rely on ordered values are inappropriate. Instead, classification models evaluate how combinations of attributes influence the probability that an observation belongs to categories such as “high risk” or “low risk,” “churn” or “retain,” or “approve” or “deny.”

Classification models can technically be applied to numeric data, but doing so ignores any natural ordering in the values. For instance, predicting age using classification treats 18 and 19 as two unrelated categories rather than recognizing that 19 represents a larger quantity than 18. As a result, while classification models *can* be used to predict numeric outcomes, they are often a poor choice when the numeric ordering carries important meaning.

Many algorithms support classification modeling, including logistic regression, decision trees, neural networks, and support vector machines. In this chapter, we focus on two foundational and widely used approaches: *logistic regression* and *classification decision trees*. Logistic regression extends linear modeling to probabilistic class prediction, while decision trees provide a flexible, non-linear alternative that does not rely on distributional assumptions such as normality. Together, these models form the

conceptual and practical foundation for more advanced techniques introduced later, including ensemble methods such as random forests and gradient boosting.

13.2Problem Setup

In a classification problem, the label is categorical rather than numeric. Instead of predicting a single numeric value (such as price), we predict which category a case belongs to (such as whether a loan ends in a good outcome or a default outcome).

Throughout this chapter, we will use a LendingClub-style dataset to predict *loan_status*. Because *loan_status* contains multiple categories, we will first recode it into a two-class label for binary classification.

Dataset overview

The dataset contains *35* columns. The table below serves as a data dictionary for every column in the file.

Table 13.1
Data dictionary for the loan classification dataset

Column	Description
loan_status	Original multi-class loan outcome label (for example, charged off, default, or other statuses).
loan_status_numeric	Numeric encoding of *loan_status* (this is a label-derived field and must not be used as a predictor).
loan_amnt	Original loan amount.
issue_d	Loan issue date (often stored as a month-year string).
term	Loan term length (for example, 36 months or 60 months).
int_rate	Interest rate on the loan.
installment	Monthly payment amount.
total_pymnt	Total payments received to date.
total_rec_prncp	Total principal received to date.
total_rec_int	Total interest received to date.
total_rec_late_fee	Total late fees received to date.
title	Loan title (free-text field, often related to purpose).
purpose	Stated purpose of the loan (categorical).
emp_title	Borrower-reported job title (free-text field).
emp_length	Borrower employment length category (often a binned or string-based category).
home_ownership	Home ownership status (for example, rent, own, mortgage).
annual_inc	Reported annual income.
verification_status	Income verification status.

Column	Description
acc_now_delinq	Number of accounts currently delinquent.
delinq_2yrs	Delinquencies in the last 2 years.
earliest_cr_line	Earliest credit line date (often stored as a month-year string).
inq_last_6mths	Credit inquiries in the last 6 months.
mths_since_last_delinq	Months since last delinquency.
mths_since_last_record	Months since last public record.
open_acc	Number of open credit accounts.
pub_rec	Number of derogatory public records.
revol_bal	Revolving credit balance.
revol_util	Revolving line utilization rate.
tot_coll_amt	Total collection amounts ever owed.
tot_cur_bal	Total current balance across accounts.
total_acc	Total number of credit lines/accounts.
total_rev_hi_lim	Total revolving high credit/limit.
dti	Debt-to-income ratio.
grade	Loan grade category.
sub_grade	More granular grade category.

Step 1: Recode the label into a two-class outcome

To make the chapter's modeling tasks clear and consistent, we will convert *loan_status* into a binary label. We will treat *Default* and *Charged Off* as the negative outcome, and we will treat all other statuses as the positive outcome.

To match the direction you requested, we will code the "good" outcome as *1* and the "bad" outcome (default or charged off) as *0*. This makes it natural to interpret predicted probabilities as "probability of a good outcome."

Step 2: Define the feature matrix X and label vector y

When building *X*, we must remove both *loan_status* and *loan_status_numeric*. These columns contain the label (or a direct encoding of the label), and including them as predictors would cause target leakage and invalidate evaluation results.

Step 3: Create train/validation/test splits (reused in later sections)

In this chapter, we will create our train/validation/test splits once and then reuse them throughout the remaining sections. We will also use *stratify* so that the class balance is similar in each split.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv(&quot;lc_small.csv&quot;)
bad_statuses = {&quot;Charged Off&quot;, &quot;Default&quot;}
df[&quot;loan_good&quot;] = (~df[&quot;loan_status&quot;].isin(bad_statuses)).astype(int)
y = df[&quot;loan_good&quot;].copy()
X = df.drop(columns=[&quot;loan_status&quot;, &quot;loan_status_numeric&quot;, &quot;loan_good&quot;]).copy()

X_train_full, X_test, y_train_full, y_test = train_test_split(
  X, y, test_size=0.20, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
  X_train_full, y_train_full, test_size=0.25, random_state=42, stratify=y_train_full
)

print(&quot;Train:&quot;, X_train.shape, &quot; Val:&quot;, X_val.shape, &quot; Test:&quot;, X_test.shape)
print(&quot;Positive class rate (loan_good=1):&quot;)
print(&quot;Train:&quot;, y_train.mean(), &quot; Val:&quot;, y_val.mean(), &quot; Test:&quot;, y_test.mean())


# Output:
# Train: (6285, 33)  Val: (2095, 33)  Test: (2096, 33)
# Positive class rate (loan_good=1):
# Train: 0.9143993635640414  Val: 0.9140811455847255  Test: 0.9141221374045801
```

From this point forward, the rest of the chapter will treat *X_train*, *X_val*, *X_test*, *y_train*, *y_val*, and *y_test* as “already created.” This keeps later sections shorter and emphasizes that good evaluation depends on a consistent split strategy.

Why stratified splitting matters

We use *stratified* splitting to preserve the proportion of default and non-default loans in the training, validation, and test sets.

This is essential in credit-risk modeling because defaults are typically rare. Without stratification, one split might contain far fewer default cases than another, leading to unstable model training and misleading performance metrics.

In the next section, we will introduce classification performance metrics and use these same data objects to evaluate our first models.

13.3Logistic Regression

The first core algorithm we study for classification is *logistic regression*. Despite its name, logistic regression is not a regression model in the usual sense—it is a probabilistic classification model designed specifically for predicting categorical outcomes.

Logistic regression is widely used in credit risk modeling, fraud detection, medical diagnosis, marketing response prediction, and many other business and scientific applications because it is fast, stable, and highly interpretable.

In this section, we use the same training, validation, and test splits created in the Problem Setup section to build and evaluate a logistic regression classifier for predicting whether a loan ends in a good outcome (*loan_good = 1*) or a bad outcome (*loan_good = 0*).

Why Linear Regression Fails for Classification

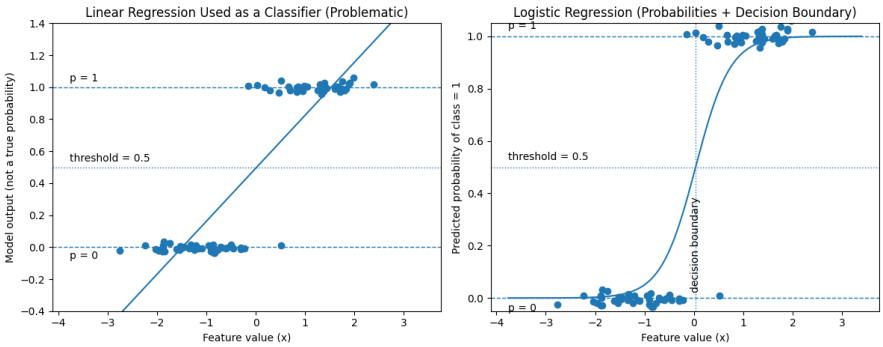
It may be tempting to apply linear regression directly to classification problems by coding classes as 0 and 1. While this sometimes produces usable predictions, the approach is fundamentally flawed.

Unbounded predictions: Linear regression can produce values below 0 or above 1, which cannot be interpreted as probabilities.

Non-probabilistic output: Linear regression does not model class probabilities in a principled way, making it difficult to interpret prediction confidence.

Poor decision boundaries: Linear regression minimizes squared error rather than classification error, often resulting in suboptimal class separation.

Classification models are explicitly designed to avoid these issues by modeling probabilities directly and constraining predictions to lie between 0 and 1.



The Logistic Function and Odds

Logistic regression solves these problems by passing a linear combination of the input features through the *logistic (sigmoid) function*.

The sigmoid function has an S-shaped curve that maps any real number into the interval (0, 1), making it ideal for modeling probabilities.

Conceptually, logistic regression models the *log-odds* of the positive class as a linear function of the predictors, and then converts those log-odds into a probability.

A predicted probability of 0.80 means the model believes there is an 80% chance that the loan ends in a good outcome, given the observed features.

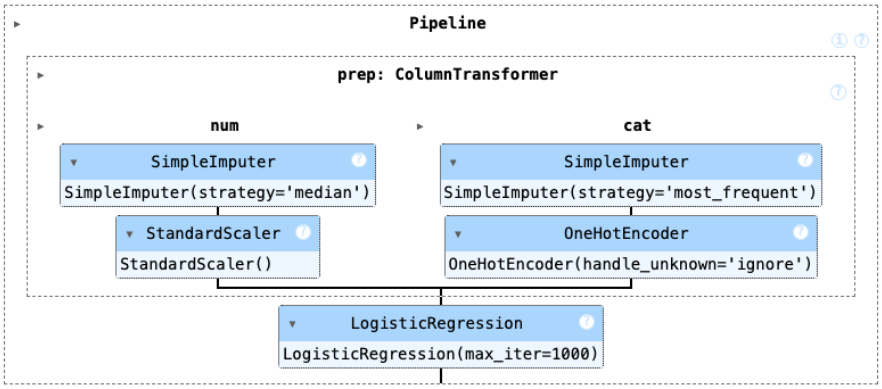
This probabilistic interpretation is one of the main reasons logistic regression is so widely used in business decision systems.

Logistic Regression in scikit-learn

We now train a logistic regression classifier using scikit-learn. As with previous chapters, we use a pipeline to combine preprocessing and modeling into a single reusable object.

This model will be reused later for evaluation and comparison with decision trees.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

num_cols = X_train.select_dtypes(include=[&quot;int64&quot;, &quot;float64&quot;]).columns.tolist()
cat_cols = X_train.select_dtypes(include=[&quot;object&quot;]).columns.tolist()

numeric_pipe = Pipeline(steps=[
  (&quot;imputer&quot;, SimpleImputer(strategy=&quot;median&quot;)),
  (&quot;scaler&quot;, StandardScaler())
])

categorical_pipe = Pipeline(steps=[
  (&quot;imputer&quot;, SimpleImputer(strategy=&quot;most_frequent&quot;)),
  (&quot;onehot&quot;, OneHotEncoder(handle_unknown=&quot;ignore&quot;))
])

preprocessor = ColumnTransformer(
  transformers=[
    (&quot;num&quot;, numeric_pipe, num_cols),
    (&quot;cat&quot;, categorical_pipe, cat_cols)
  ]
)

logit_model = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;clf&quot;, LogisticRegression(max_iter=1000))
])

logit_model.fit(X_train, y_train)
```



Logistic regression provides two types of predictions: class labels and class probabilities.

predict() returns the predicted class (0 or 1).

predict_proba() returns the estimated probability for each class.

```
y_val_pred = logit_model.predict(X_val)
y_val_proba = logit_model.predict_proba(X_val)

print(&quot;First 5 predicted classes:&quot;, y_val_pred[:5])
print(&quot;First 5 predicted probabilities (P(class=1)):&quot;, y_val_proba[:5, 1])


# Output:
# First 5 predicted classes: [1 1 1 1 1]
# First 5 predicted probabilities (P(class=1)): [0.99965759 0.99650636 0.99905931 0.99639334 0.99999159]
```

Interpreting Logistic Regression Coefficients

Each coefficient in a logistic regression model represents the change in the *log-odds* of the positive class for a one-unit increase in the corresponding feature.

Exponentiating a coefficient converts it into an *odds ratio*, which is often easier to interpret.

An odds ratio greater than 1 indicates higher odds of a good outcome, while a value less than 1 indicates lower odds.

In predictive modeling, coefficients are interpreted as *associations useful for prediction*, not as causal effects.

This differs from causal regression, where coefficient interpretation requires strong assumptions about confounding, functional form, and omitted variables.

In classification modeling, the primary purpose of coefficients is to understand model behavior and feature influence, not to establish cause-and-effect relationships.

Strengths and Weaknesses of Logistic Regression

- Strengths
  - Simple and mathematically well-understood.
  - Produces calibrated probabilities.
  - Highly interpretable coefficients.
  - Computationally efficient and stable.
  - Works well as a baseline classifier.
- Weaknesses

- Assumes a linear decision boundary in feature space.
- Struggles with complex nonlinear relationships.
- Requires careful feature engineering to capture interactions.
- Performance may lag behind tree-based models on complex datasets.

Despite these limitations, logistic regression remains one of the most important and widely deployed classification algorithms in practice. In the next section, we will study classification decision trees and compare their behavior to logistic regression.

Regression assumptions: linear vs logistic

Because logistic regression is derived from linear regression, it inherits some assumptions but modifies or removes others. Understanding which assumptions still apply is important for knowing when logistic regression is appropriate and how to diagnose problems in practice.

Table 13.2
Comparison of linear regression and logistic regression assumptions

Assumption	Linear regression	Logistic regression	Explanation
Linearity of relationship	Required between X and Y	Required between X and log-odds	Logistic regression does not assume a linear relationship between predictors and the probability itself, but it does assume linearity between predictors and the log-odds of the outcome.
Independence of observations	Required	Required	Each observation should represent an independent case. Violations occur with repeated measurements, clustered data, or time-series dependence.
No perfect multicollinearity	Required	Required	Highly correlated predictors make coefficient estimates unstable and difficult to interpret in both models.
Normality of residuals	Required for inference	Not required	Logistic regression does not model residuals as normally distributed because the outcome is categorical rather than continuous.
Homoscedasticity	Required for inference	Not required	The variance of a binary outcome is determined by its probability, so constant variance is neither expected nor assumed.
Bounded predictions	Not guaranteed	Guaranteed	Logistic regression enforces predictions between 0 and 1 by construction using the logistic function.
Correct functional form	Important	Important	Both models benefit from appropriate feature engineering (polynomials, interactions, transformations) to capture nonlinear structure.

The most important conceptual difference is that logistic regression replaces the assumption of normally distributed errors with a probabilistic model for binary outcomes. This makes it well suited for classification while preserving many of the interpretability benefits of linear regression.

In practice, this means that many of the same diagnostic habits you developed for linear regression—checking for multicollinearity, inspecting influential predictors, and validating functional form—remain useful for logistic regression, even though the error structure and interpretation of coefficients change.

13.4Decision Tree Classification

In the prior chapter, you learned how regression trees predict a numeric outcome by splitting the feature space into regions and predicting the average label value inside each leaf.

In this section, we apply the same core idea to a new goal: predicting a *class label* (such as whether a loan will end in a negative outcome). A *classification tree* learns a sequence of splits that separates classes as cleanly as possible.

Because decision trees use simple "if–then" rules, they are intuitive and easy to visualize. They also require fewer modeling assumptions than many other algorithms. However, like regression trees, they can overfit easily if we let them grow without constraints.

How classification trees differ from regression trees

Regression trees and classification trees are built using the same greedy splitting strategy: at each node, the algorithm searches for the split that most improves a chosen objective function. The key difference is the objective function itself.

A regression tree chooses splits that reduce numeric error (often measured with *mean squared error*). A classification tree chooses splits that reduce *class impurity*, which measures how mixed the classes are inside a node.

Table 13.3
Regression trees vs. classification trees

Concept	Regression tree	Classification tree
Label type	Numeric (continuous)	Categorical (class labels)
Split objective	Reduce numeric error (e.g., MSE)	Reduce class impurity (e.g., Gini or entropy)
Leaf contents	Mean (or median) of y in the leaf	Class proportions in the leaf
Prediction rule	Numeric value (mean/median)	Majority vote (most common class)
Probability output	Not a probability (unless transformed later)	Often a class probability estimate from leaf proportions

Impurity: what the tree tries to reduce

In a classification node, the data consists of a mix of class labels. A node is considered "pure" if almost all observations belong to the same class, and "impure" if the classes are heavily mixed.

Two common impurity measures are *Gini impurity* and *entropy*. Both reach their minimum when a node is perfectly pure and their maximum when classes are evenly mixed.

In scikit-learn, you choose this criterion using the *criterion* hyperparameter. For binary classification, the most common options are *gini* (default) and *log_loss* (or *entropy*, depending on the version).

What a leaf means in classification

In a regression tree, a leaf contains a numeric prediction such as the average sale price in that region. In a classification tree, a leaf contains a *class distribution*.

For example, suppose a leaf contains 100 loans, and 25 of them ended in a negative outcome. Then the leaf's estimated probability for the negative class is 0.25, and the probability for the positive class is 0.75.

When you call *predict*, the tree assigns each observation to its leaf and returns the most common class in that leaf (majority vote). When you call *predict_proba*, the tree returns the class probabilities implied by leaf proportions.

Practical takeaway

Conceptually, classification trees work the same way as regression trees: repeated splits build a hierarchy of decision rules. The main change is that the tree is now optimizing purity (Gini/entropy/log loss) instead of numeric error.

Next, we will train a *DecisionTreeClassifier* on our loan dataset using a predictive workflow and examine how hyperparameters like *max_depth* and *min_samples_leaf* control model complexity.

Training a Classification Tree in Python

We now train a classification tree using the same loan dataset introduced earlier in this chapter. The workflow mirrors what you have already seen for regression trees: train/test split, preprocessing pipeline, model fitting, and evaluation.

In scikit-learn, classification trees are implemented with the *DecisionTreeClassifier* class.

Key hyperparameters

- *max_depth*: Maximum number of splits from root to leaf. Controls overall complexity.
- *min_samples_leaf*: Minimum number of observations allowed in a leaf node.
- *criterion*: Impurity measure used to choose splits (*gini*, *entropy*, or *log_loss*).

As before, we begin with a simple model using mostly default settings and then study how performance changes as we adjust complexity.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, log_loss

tree_clf = DecisionTreeClassifier(
  max_depth=None,
  min_samples_leaf=1,
  criterion=&quot;gini&quot;,
  random_state=27
)

model = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;tree&quot;, tree_clf)
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
acc = accuracy_score(y_test, y_pred)
ll = log_loss(y_test, y_prob)

print(&quot;Test accuracy:&quot;, round(acc, 4))
print(&quot;Test log loss:&quot;, round(ll, 4))


# Output:
# Test accuracy: 0.9165
# Test log loss: 3.0094
```

Accuracy measures the fraction of correct class predictions. Log loss evaluates the quality of predicted probabilities and penalizes confident mistakes more strongly.

These two metrics often move in different directions as model complexity changes, which becomes important when diagnosing overfitting. We will explain them in more detail later.

Next, we will systematically vary the tree depth to visualize how classification trees overfit and how validation performance reveals the optimal level of complexity.

Overfitting in Classification Trees

Classification trees are highly flexible models. This flexibility allows them to capture complex patterns, but it also makes them especially prone to *overfitting*.

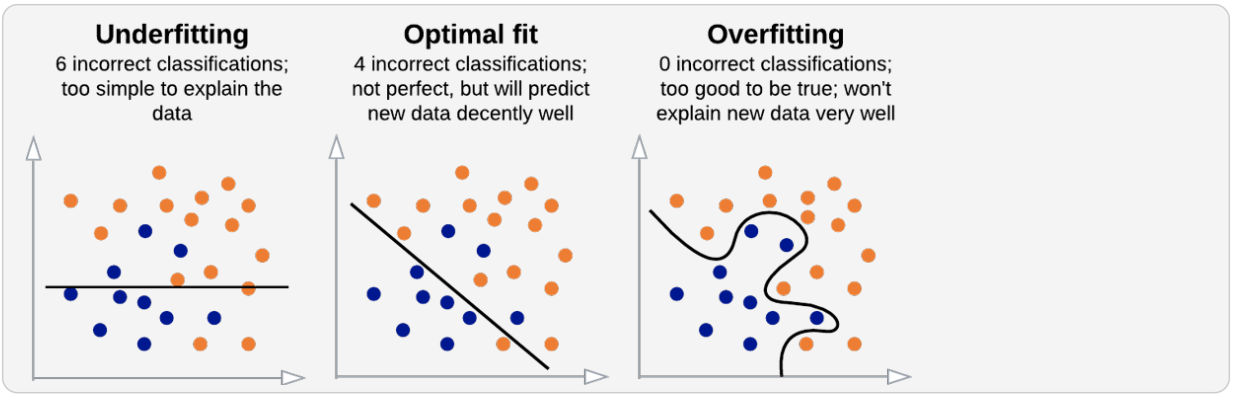
An overfit classification tree memorizes the training data, achieving very high training accuracy while performing substantially worse on new, unseen observations.

Why depth causes overfitting

Each additional level of depth allows the tree to create more specialized decision rules. Eventually, the model begins splitting on noise rather than signal.

This produces leaves that contain only a few observations and predictions that are unstable across samples.

The figure below illustrates this progression using a simple two-class example. On the left, the decision boundary is nearly flat—too simple to separate the classes effectively. This is **underfitting**: the model ignores meaningful structure in the data. In the center, the boundary captures the general pattern without chasing every individual point. This is the **optimal fit**: a few training observations are misclassified, but the model will generalize well to new data. On the right, the boundary twists around every point, achieving zero training errors. This is **overfitting**: the model has memorized the training data, including its noise, and will perform poorly on observations it has not seen before.



As you work through the depth-sweep experiment below, keep this visual in mind. Increasing tree depth moves the model from left to right in the figure: shallow trees underfit, moderate-depth trees approximate the optimal boundary, and excessively deep trees overfit.

Depth sweep experiment

To diagnose overfitting, we train multiple trees with increasing values of *max_depth* and compare performance on both the training and validation sets.

We track two metrics:

- *Accuracy*, which measures classification correctness.
- *Log loss*, which measures probability quality and penalizes confident errors.

```
from sklearn.metrics import accuracy_score, log_loss
import pandas as pd

results = []

for depth in [1, 2, 3, 4, 6, 8, 10, 12, 15]:
  tree_clf = DecisionTreeClassifier(
    max_depth=depth,
    min_samples_leaf=1,
    criterion=&quot;gini&quot;,
    random_state=27
  )

  pipe = Pipeline(steps=[
    (&quot;prep&quot;, preprocessor),
    (&quot;tree&quot;, tree_clf)
  ])
```

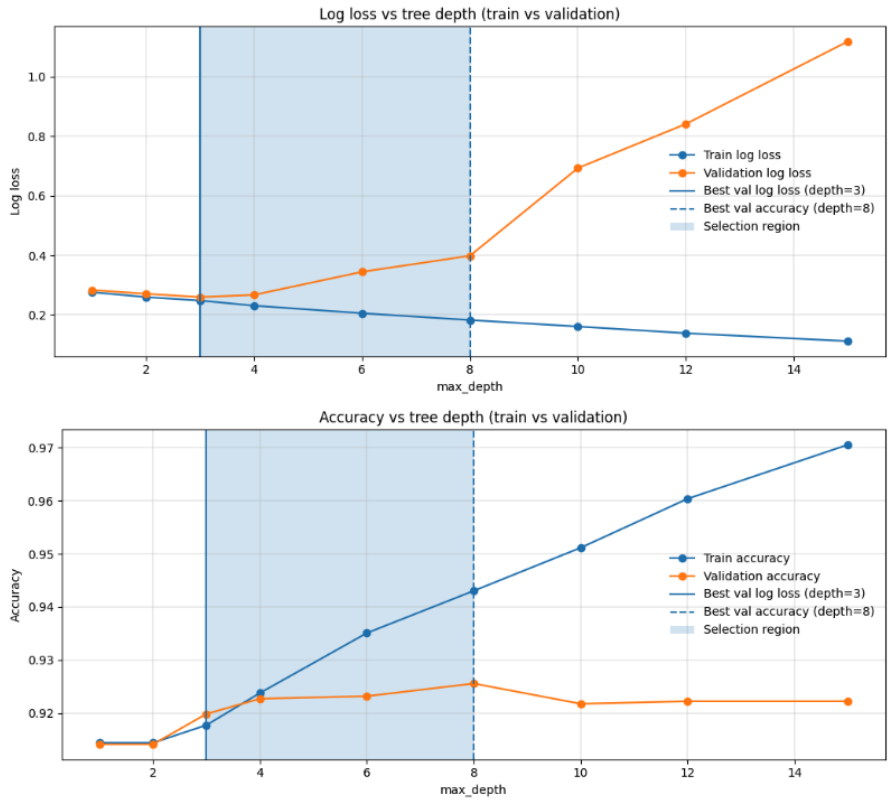
```
    pipe.fit(X_train, y_train)
    y_train_pred = pipe.predict(X_train)
    y_train_prob = pipe.predict_proba(X_train)[:, 1]
    y_val_pred = pipe.predict(X_val)
    y_val_prob = pipe.predict_proba(X_val)[:, 1]

    results.append({
      &quot;max_depth&quot;: depth,
      &quot;train_accuracy&quot;: accuracy_score(y_train, y_train_pred),
      &quot;val_accuracy&quot;: accuracy_score(y_val, y_val_pred),
      &quot;train_log_loss&quot;: log_loss(y_train, y_train_prob),
      &quot;val_log_loss&quot;: log_loss(y_val, y_val_prob)
    })

depth_results = pd.DataFrame(results)
depth_results
```

	max_depth	train_accuracy	val_accuracy	train_log_loss	val_log_loss
0	1	0.914399	0.914122	0.276132	0.283294
1	2	0.914399	0.914122	0.259744	0.270788
2	3	0.917741	0.919847	0.248141	0.259777
3	4	0.923787	0.922710	0.230856	0.267129
4	6	0.935084	0.923187	0.205455	0.344557
5	8	0.943039	0.925573	0.182615	0.398682
6	10	0.951154	0.921756	0.160869	0.693329
7	12	0.960382	0.922233	0.138391	0.841523
8	15	0.970565	0.922233	0.111732	1.118724

The resulting table summarizes how classification performance evolves as the tree becomes deeper. But let's make this a bit easier to process by visualizing it below:

```
import matplotlib.pyplot as plt

best_ll_depth = int(depth_results.loc[depth_results[&quot;val_log_loss&quot;].idxmin(), &quot;max_depth&quot;])
best_acc_depth = int(depth_results.loc[depth_results[&quot;val_accuracy&quot;].idxmax(), &quot;max_depth&quot;])
left = min(best_ll_depth, best_acc_depth)
right = max(best_ll_depth, best_acc_depth)
x = depth_results[&quot;max_depth&quot;].to_numpy()
plt.figure(figsize=(10.5, 4.8))
plt.plot(x, depth_results[&quot;train_log_loss&quot;], marker=&quot;o&quot;, linewidth=1.5, label=&quot;Train log
loss&quot;)
plt.plot(x, depth_results[&quot;val_log_loss&quot;], marker=&quot;o&quot;, linewidth=1.5, label=&quot;Validation log
loss&quot;)
plt.axvline(best_ll_depth, linewidth=1.5, label=f&quot;Best val log loss (depth={best_ll_depth})&quot;)
plt.axvline(best_acc_depth, linewidth=1.5, linestyle=&quot;--&quot;, label=f&quot;Best val accuracy (depth=
{best_acc_depth})&quot;)
plt.axvspan(left, right, alpha=0.18, label=&quot;Selection region&quot;)
plt.xlabel(&quot;max_depth&quot;)
plt.ylabel(&quot;Log loss&quot;)
plt.title(&quot;Log loss vs tree depth (train vs validation)&quot;)
plt.legend(frameon=False)
plt.grid(True, linewidth=0.3)
plt.tight_layout()
plt.show()
plt.figure(figsize=(10.5, 4.8))
plt.plot(x, depth_results[&quot;train_accuracy&quot;], marker=&quot;o&quot;, linewidth=1.5, label=&quot;Train
accuracy&quot;)
plt.plot(x, depth_results[&quot;val_accuracy&quot;], marker=&quot;o&quot;, linewidth=1.5, label=&quot;Validation
accuracy&quot;)
plt.axvline(best_ll_depth, linewidth=1.5, label=f&quot;Best val log loss (depth={best_ll_depth})&quot;)
plt.axvline(best_acc_depth, linewidth=1.5, linestyle=&quot;--&quot;, label=f&quot;Best val accuracy (depth=
{best_acc_depth})&quot;)
plt.axvspan(left, right, alpha=0.18, label=&quot;Selection region&quot;)
plt.xlabel(&quot;max_depth&quot;)
```

```
plt.ylabel(&quot;Accuracy&quot;)
plt.title(&quot;Accuracy vs tree depth (train vs validation)&quot;)
plt.legend(frameon=False)
plt.grid(True, linewidth=0.3)
plt.tight_layout()
plt.show()
```



To choose a reasonable value for *max_depth*, we evaluate candidate depths using *validation-set* metrics rather than training-set metrics. The training curves almost always improve as depth increases, because deeper trees can memorize the training data.

In the plots, the solid vertical line marks the depth with the lowest *validation log loss*. Log loss uses the full predicted probabilities and penalizes overconfident mistakes, so it is usually the most reliable criterion when you care about probability quality (for example, risk scoring and decision-making under uncertainty).

The dashed vertical line marks the depth with the highest *validation accuracy*. Accuracy evaluates only the final class label after applying a probability threshold (typically 0.50), so it can be easier to explain but it ignores confidence and can be sensitive to class imbalance.

The shaded region spans the depths between the best validation log loss and the best validation accuracy. Any depth inside this interval is a defensible choice, and selecting within it becomes a modeling judgment: choose closer to the log-loss optimum when calibrated probabilities matter, or closer to the accuracy optimum when the primary goal is correct hard-label decisions.

A practical rule is to pick the *simplest* depth inside the shaded region, because simpler trees are usually more stable and easier to explain while still achieving near-optimal validation performance.

Modeling judgment
If your application uses predicted probabilities to make decisions (for example, pricing, risk scores, or ranking), prioritize *validation log loss* and select a depth near the solid line. If your application only needs a yes/no decision and the threshold is fixed, accuracy may be a meaningful secondary consideration. When the two best depths differ, treat the shaded region as the set of "reasonable" options, then choose the simplest tree in that region unless you have a clear business reason to do otherwise.

Bias–variance tradeoff revisited

The validation curves above illustrate a classic concept you first encountered in linear regression: the *bias–variance tradeoff*. Tree depth directly controls where a model falls on this spectrum.

Table 13.4
Tree depth and the bias–variance tradeoff

Tree depth	Primary risk	Typical behavior
Very shallow	High bias (underfitting)	Misses important patterns; low training and validation performance
Moderate	Balanced	Best generalization; near-optimal validation accuracy and log loss
Very deep	High variance (overfitting)	Excellent training performance but degrading validation performance

The shaded selection region in the validation plots corresponds to the middle row of this table: depths where the model is complex enough to capture structure but not so complex that it memorizes noise.

In practice, modelers identify this region using validation curves or cross-validation, then choose the simplest model within it that satisfies performance requirements.

In the next section, we will visualize the learned tree structure to better understand how classification decisions are formed inside the model.

13.5Visualizing Classification Trees

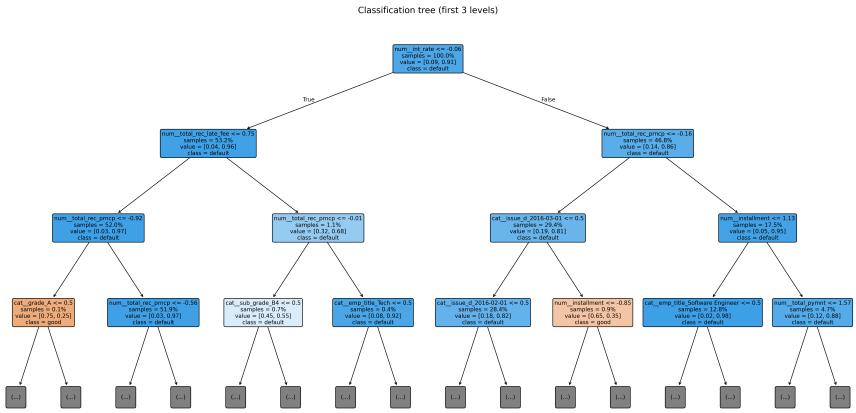
Decision trees are often described as "interpretable" models because their structure can be visualized directly. In this section, we examine how to plot a trained classification tree and how to interpret the information shown inside each node.

Unlike linear and logistic regression models, where predictions are produced through equations, a tree model makes decisions by following a sequence of splits from the root to a leaf. Visualizing this process helps clarify how the model converts input features into class predictions.

Plotting a classification tree in Python

We will visualize the trained classification tree from the previous section using *sklearn.tree.plot_tree*. Because large trees quickly become unreadable, we typically visualize only shallow trees or pruned versions of deeper models.

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

tree_model = model.named_steps[&quot;tree&quot;]
plt.figure(figsize=(20, 10), dpi=350)

plot_tree(
  tree_model,
  feature_names=preprocessor.get_feature_names_out(),
  class_names=[&quot;default&quot;, &quot;good&quot;],
  filled=True,
  rounded=True,
  max_depth=3,
  fontsize=9,
  precision=2,
  proportion=True,
  impurity=False
)

plt.title(&quot;Classification tree (first 3 levels)&quot;, fontsize=14)
plt.tight_layout()
plt.show()
```



The visualization is controlled by several important parameters in *plot_tree*. *feature_names* supplies readable variable names for each split, and *class_names* labels the predicted categories shown in leaf nodes. Setting *filled=True* colors nodes by the predicted class and class purity, while *rounded=True* improves visual clarity by softening box corners. The *max_depth* parameter limits how many levels of the tree are drawn, which is useful for focusing on high-level structure without overwhelming detail. *fontsize* controls text size, *precision* limits the number of decimal places displayed in probabilities, and *proportion=True* shows class probabilities instead of raw sample counts. Setting *impurity=False* removes Gini or entropy values to reduce clutter. Additional optional parameters include *label* (to control how much text is shown in nodes), *node_ids* (to display internal node numbers for debugging or pruning), and *rotate* (to draw the tree horizontally for very wide models).

What each node represents

Each rectangle in the tree visualization is called a *node*. Internal nodes represent decision points, while leaf nodes represent final predictions.

- *Predicted class*: The class label shown at the bottom of the node is the majority class among training samples that reached that node.
- *Class probabilities*: The value vector shows the proportion of samples from each class at that node (for example, 0.85 good and 0.15 default).
- *Impurity* (optional): The Gini or entropy value measures how mixed the classes are at that node. Lower values indicate purer nodes. Our code sets *impurity=False* to reduce clutter, but you can enable it by removing that parameter.
- *Sample count*: The number of training observations that reached the node.
- *Split rule*: For internal nodes, the condition (for example, income ≤ 42,000) determines which branch each observation follows.

How predictions are formed

To classify a new observation, the tree starts at the root node and evaluates the split rule. The observation follows the corresponding branch until it reaches a leaf node, where the predicted class is produced.

Although the final prediction is a single class label, the model internally computes class probabilities based on the proportion of training samples in the leaf.

Interpretation limits of tree visualizations

Tree visualizations become difficult to interpret as depth increases. Even moderately sized trees can contain dozens or hundreds of nodes, making global reasoning about the model impractical.

In addition, splits are chosen greedily. A feature that appears near the top of the tree is not necessarily globally more important than all features below it; it simply provided the best local impurity reduction at that step.

Why shallow trees are often preferable

Shallow trees are often preferred when interpretability is important. A tree with depth 3 or 4 can usually be inspected and explained by a human, while deeper trees behave more like opaque predictive machines.

From a predictive perspective, shallow trees also tend to generalize better because they avoid memorizing idiosyncratic patterns in the training data.

For these reasons, tree depth is commonly restricted using hyperparameters such as *max_depth* and *min_samples_leaf*, or by pruning after training.

In practice, tree visualizations are best used as diagnostic and explanatory tools rather than complete descriptions of a model's behavior.

Next, we extend classification trees to problems involving more than two outcome categories.

Saving high-resolution tree diagrams for large models

When trees become larger, the default screen resolution is often insufficient to read split rules, class probabilities, and impurity values. In these cases, it is best to save the tree visualization as a high-resolution image file using the *dpi* parameter in Matplotlib.

Below, we first train and export a tree at a depth suggested by the sweep (max depth = 4), and then export a deeper diagnostic tree (max depth = 8) using a much higher DPI suitable for zooming or printing.

```
# Retrain with the selected depth from the sweep
selected_tree = DecisionTreeClassifier(
  max_depth=4, min_samples_leaf=1,
  criterion=&quot;gini&quot;, random_state=27
)
selected_model = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor), (&quot;tree&quot;, selected_tree)
])
selected_model.fit(X_train, y_train)

plt.figure(figsize=(20, 10), dpi=250)
plot_tree(
  selected_model.named_steps[&quot;tree&quot;],
  feature_names=preprocessor.get_feature_names_out(),
  class_names=[&quot;default&quot;, &quot;good&quot;],
  filled=True,
  rounded=True,
  fontsize=9,
  precision=2,
  proportion=True,
  impurity=False
)

plt.title(&quot;Classification tree (max_depth = 4)&quot;, fontsize=14)
plt.tight_layout()
plt.savefig(&quot;classification_tree_depth4.png&quot;, bbox_inches=&quot;tight&quot;)
plt.close()
```

The file *classification_tree_depth4.png* preserves the full structure of the selected model while remaining readable at standard zoom levels.

```
deep_tree = DecisionTreeClassifier(
  max_depth=8,
  min_samples_leaf=1,
  criterion=&quot;gini&quot;,
  random_state=27
)

deep_model = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;tree&quot;, deep_tree)
```

```
])

deep_model.fit(X_train, y_train)
plt.figure(figsize=(28, 16), dpi=450)

plot_tree(
  deep_model.named_steps[&quot;tree&quot;],
  feature_names=preprocessor.get_feature_names_out(),
  class_names=[&quot;default&quot;, &quot;good&quot;],
  filled=True,
  rounded=True,
  fontsize=8,
  precision=2,
  proportion=True,
  impurity=False
)

plt.title(&quot;Classification tree (max_depth = 8)&quot;, fontsize=16)
plt.tight_layout()
plt.savefig(&quot;classification_tree_depth8_highdpi.png&quot;, bbox_inches=&quot;tight&quot;)
plt.close()
```

The file *classification_tree_depth8_highdpi.png* is saved at very high resolution so that individual nodes remain legible even when the tree contains many dozens of splits.

Practical tip
For trees deeper than 5–6 levels, visualization is primarily useful for debugging and teaching rather than interpretation. In applied modeling, feature importance measures and validation metrics usually provide more reliable insight than attempting to reason about hundreds of individual decision paths.

Table 13.5
Recommended DPI settings for exporting decision tree diagrams

Use case	Recommended DPI	Notes
On-screen viewing (notebooks, slides)	150–200	Sufficient for readability on standard displays and small trees.
Textbooks and PDF reports	250–350	Balances file size and clarity when zooming or printing.
Large trees (depth ≥ 7)	400–500	Preserves legibility of split rules and probabilities in dense diagrams.
Posters and print layouts	450–600	Recommended when diagrams will be enlarged physically.

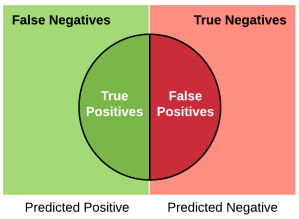
13.6Evaluation Metrics for Classification

After training a classification model, the next question is not “How accurate is it?” but “How costly are its mistakes?” In a business setting, classification errors often have unequal consequences, so selecting evaluation metrics is fundamentally a decision about tradeoffs.

In the Lending Club example, our goal is to predict whether a loan ends in *default* (the bad outcome) versus *good standing* (all other outcomes). This section shows how to evaluate a model using both (1) *threshold-based* metrics (based on predicted classes) and (2) *probability-based* metrics (based on predicted probabilities).

Metric choice is a cost choice: Use *accuracy* when false positives and false negatives have similar costs, *precision* when false positives are costly (approving too many risky loans as good), *recall* when false negatives are costly (rejecting too many creditworthy borrowers), and *F1* when you need a single number that balances precision and recall.

In the code examples that follow, we will reuse the Lending Club train/test objects and preprocessing pipeline created most recently in the prior section, and we will evaluate a *DecisionTreeClassifier* with *max_depth = 8*, which also happens to be in the range of valid potential options.



Confusion Matrix		Predicted	
		Negative	Positive
Actual Labels	Negative	True Negative	False Positive
	Positive	False Negative	True Positive

Figure 13.1: Conceptual Confusion Matrix

We begin with the confusion matrix, because it makes every other metric easier to understand.

Baseline Accuracy and Class Imbalance

Before interpreting any accuracy value, always compute the *baseline accuracy* you would get from a "no-skill" model that predicts only the most common class. In a two-class problem, the baseline is simply the larger class proportion: if 75% of loans are *good* and 25% are *default*, then a useless classifier that predicts *good* for every case achieves 75% accuracy without learning anything.

This is why accuracy can be misleading in imbalanced datasets: a model can appear "high accuracy" while still failing to detect the minority class that matters most (such as defaults). A quick case example: imagine a lender screens 10,000 loans, and only 500 (5%) end in default. A model that predicts *no default* for everyone achieves 95% accuracy, but its recall for defaults is 0.00 because it never catches any risky borrowers.

Whenever your model's accuracy is near the baseline accuracy, it is not providing meaningful predictive value. Your goal is not merely to exceed baseline accuracy, but to improve the metrics that reflect your business costs (often recall, precision, and probability quality). The baseline concept also connects directly to the confusion matrix: a majority-class classifier produces a confusion matrix with a large true-negative count, but *zero true positives* for the minority class.

For that reason, always evaluate imbalanced classification models with metrics that focus on the positive class (precision, recall, F1) and with probability-based metrics (log loss), not accuracy alone. In

multiclass problems, the same idea applies: the baseline accuracy is the proportion of the most frequent class. If one class dominates, a model can earn deceptively high accuracy by predicting only that class.

Next, we return to the confusion matrix because it makes the baseline and all other metrics easy to interpret.

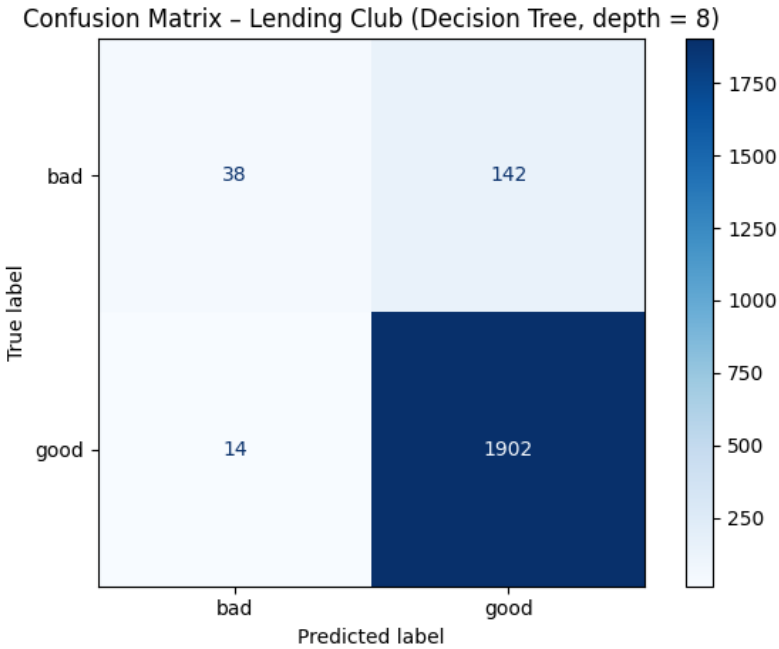
Confusion Matrix

The confusion matrix A table that compares predicted classes to actual classes to summarize all correct and incorrect classification outcomes. is the foundation for nearly all classification metrics. For a two-class problem, it contains four values: true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP).

Because *loan_good = 1* is the positive class, the confusion matrix maps directly to lending decisions. A *true positive (TP)* is a good loan correctly approved. A *false positive (FP)* is a default incorrectly approved as good—the most expensive error. A *false negative (FN)* is a good loan incorrectly flagged as risky—a missed business opportunity. A *true negative (TN)* is a default correctly rejected.

The code below uses the trained decision tree classifier (*max_depth = 8*) and the Lending Club test set created earlier in the chapter.

```
from sklearn import metrics
import matplotlib.pyplot as plt

# Class predictions on test set
y_test_pred = deep_model.predict(X_test)

# Confusion matrix (explicit label order)
cm = metrics.confusion_matrix(
  y_test,
  y_test_pred,
  labels=[0, 1] # Use numeric labels as they are in y_test
)

cm_display = metrics.ConfusionMatrixDisplay(cm, display_labels=[&quot;bad&quot;, &quot;good&quot;])
cm_display.plot(values_format=&quot;d&quot;, cmap=&quot;Blues&quot;)
plt.title(&quot;Confusion Matrix – Lending Club (Decision Tree, depth = 8)&quot;)
plt.tight_layout()
plt.show()
```



Cells on the diagonal represent correct predictions (true negatives and true positives). Off-diagonal cells represent errors: false positives (predicting *default* for a good loan) and false negatives (predicting *good* for a loan that actually defaults).

Every metric introduced next is computed directly from these four numbers.

Accuracy

Accuracy The proportion of all predictions that are correct: (TP + TN) / (TP + TN + FP + FN). is the simplest summary metric derived from the confusion matrix.

Accuracy answers the question: *"What fraction of loans did the model classify correctly overall?"*

In scikit-learn, accuracy can be computed either with the *accuracy_score* function or with the model's built-in *.score()* method. Both produce the same value when applied to classification models.

```
from sklearn.metrics import accuracy_score

acc1 = accuracy_score(y_test, y_test_pred)
acc2 = deep_model.score(X_test, y_test)

print(&quot;Accuracy (accuracy_score):&quot;, round(acc1, 4))
print(&quot;Accuracy (deep_model.score):&quot;, round(acc2, 4))
# Accuracy (accuracy_score): 0.9275
# Accuracy (deep_model.score): 0.9275
```

Because accuracy treats all errors equally, it is most appropriate when false positives and false negatives have similar business costs and when class sizes are reasonably balanced.

In credit risk modeling, this assumption is rarely true: approving a loan that later defaults (false negative) is usually far more costly than rejecting a good borrower (false positive). For this reason, accuracy alone is not sufficient for evaluating lending models.

The next metrics—precision, recall, and F1—separate different types of mistakes and allow us to measure these risks explicitly.

Precision and Recall

Precision The proportion of predicted positive cases that are truly positive: TP / (TP + FP). answers the question: *"When the model predicts a loan is good, how often is it correct?"* High precision means few defaults slip through as approved loans.

Recall The proportion of actual positive cases that are correctly identified: TP / (TP + FN). answers the question: *"Of all loans that are actually good, how many did the model correctly approve?"*

Precision focuses on avoiding false alarms (false positives), while recall focuses on avoiding missed detections (false negatives). These two goals often conflict.

In the Lending Club context, the positive class is *loan_good = 1*. This means:

- **False positive (FP):** A defaulting borrower incorrectly approved as good—a direct financial loss.
- **False negative (FN):** A creditworthy borrower incorrectly flagged as risky—a missed business opportunity.

In most lending applications, false positives are the most expensive errors because an approved default leads directly to financial losses. This makes precision especially important, though high recall is also desirable to avoid turning away too many creditworthy borrowers.

```
from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_test, y_test_pred)
recall = recall_score(y_test, y_test_pred)

print(&quot;Precision:&quot;, round(precision, 4))
print(&quot;Recall:   &quot;, round(recall, 4))


# Output:
# Precision: 0.9305
# Recall:    0.9927
```

A model can achieve high precision by predicting *good* only when very confident, reducing the chance of approving a default. But this conservative approach often lowers recall, rejecting more creditworthy

borrowers in the process.

Conversely, a model can achieve high recall by approving loans more liberally, capturing nearly all creditworthy borrowers. But this usually lowers precision, letting more defaults slip through.

This tension motivates the F1-score, which combines precision and recall into a single metric.

F1 Score and the Classification Report

F1-score The harmonic mean of precision and recall: 2 × (precision × recall) / (precision + recall). provides a single number that balances false positives and false negatives.

The harmonic mean penalizes extreme values. If either precision or recall is low, the F1-score will also be low. This makes F1 useful when both types of errors matter.

In credit risk modeling, F1 is often preferred over accuracy because it balances the cost of approving bad loans (false positives) against the cost of rejecting good borrowers (false negatives).

```
from sklearn.metrics import f1_score

f1 = f1_score(y_test, y_test_pred)

print(&quot;F1 score:&quot;, round(f1, 4))


# Output:
# F1 score: 0.9606
```

Rather than computing each metric separately, scikit-learn can generate a full summary table called the *classification report*.

The classification report shows precision, recall, F1-score, and support (number of observations) for each class.

```
from sklearn.metrics import classification_report
import pandas as pd

report = classification_report(y_test, y_test_pred, output_dict=True)
pd.DataFrame(report).transpose()
```

	precision	recall	f1-score	support
0	0.730769	0.211111	0.327586	180.000000
1	0.930528	0.992693	0.960606	1916.000000
accuracy	0.925573	0.925573	0.925573	0.925573
macro avg	0.830649	0.601902	0.644096	2096.000000
weighted avg	0.913373	0.925573	0.906244	2096.000000

Key columns in the classification report:

- **precision** – reliability of positive predictions
- **recall** – ability to capture actual positives
- **f1-score** – balance between precision and recall
- **support** – number of true samples in each class

For imbalanced datasets like loan defaults, macro averages treat each class equally, while weighted averages account for class frequency.

Always examine per-class metrics before trusting a single summary number. A high overall score can still hide poor performance on the default class.

Log Loss and Probability Quality

log loss A probabilistic classification metric that penalizes confident wrong predictions more than uncertain predictions; lower values indicate better probability estimates. evaluates how good your *predicted probabilities* are, not just whether your predicted class label is correct.

This matters because many business decisions depend on probability confidence. For example, a loan team may treat a predicted default probability of 0.55 very differently than 0.95, even though both would be classified as “default” at a 0.50 threshold.

Log loss is especially useful when you care about calibrated risk scores. It heavily penalizes predictions like “99% chance of default” when the loan actually does not default.

In this chapter’s workflow, we chose the classification tree depth using *validation log loss*. That means our selected model is optimized for probability quality rather than raw accuracy at a single threshold.

```
from sklearn.metrics import log_loss

# Probability of the positive class (loan_good = 1)
y_test_prob = deep_model.predict_proba(X_test)[:, 1]
ll = log_loss(y_test, y_test_prob)

print(&quot;Test log loss:&quot;, round(ll, 4))
# Test log loss: 0.5716
```

Interpretation guide:

- **Lower is better.** Unlike accuracy, higher values are worse.
- Log loss is minimized when predicted probabilities match the true outcomes.
- A few extremely confident wrong predictions can raise log loss sharply.

When two models have similar accuracy, the model with better log loss is usually the better choice if your downstream decision process uses probability scores (risk tiers, pricing, prioritization, or manual review queues).

Why log loss can disagree with accuracy
Accuracy only checks whether the predicted class label matches the true label at one fixed threshold (often 0.50). Log loss uses the full probability value, so it can prefer a model that makes slightly fewer confident mistakes—even if its 0.50-threshold accuracy is similar.

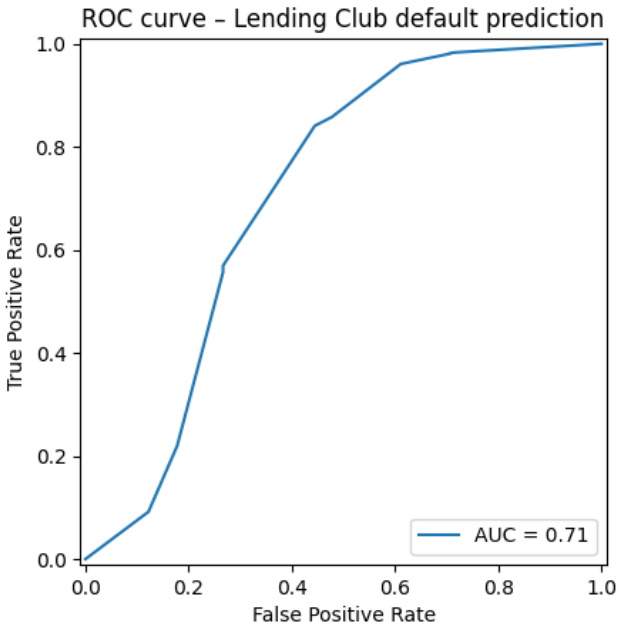
ROC Curves and AUC

Another way to evaluate a classifier across *all possible thresholds* is with the ROC curve A plot showing the tradeoff between true positive rate (recall) and false positive rate as the classification threshold varies..

The ROC curve answers the question: *How well can the model separate good loans from defaulting loans, regardless of where we place the cutoff threshold?*

Instead of using predicted class labels, ROC curves use the model's predicted probabilities. This makes them complementary to log loss, which also evaluates probability quality.

```
from sklearn.metrics import roc_curve, roc_auc_score, RocCurveDisplay
import matplotlib.pyplot as plt

y_test_prob = deep_model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_test_prob)
auc_value = roc_auc_score(y_test, y_test_prob)
disp = RocCurveDisplay(fpr=fpr, tpr=tpr, roc_auc=auc_value)
disp.plot()
plt.title(&quot;ROC curve – Lending Club default prediction&quot;)
plt.show()

print(&quot;AUC:&quot;, round(auc_value, 4))
# AUC: 0.7743
```



Each point on the ROC curve corresponds to a different probability threshold. Moving along the curve trades off:

- **True Positive Rate (TPR / Recall)**: proportion of actual good loans correctly approved.
- **False Positive Rate (FPR)**: proportion of actual defaults incorrectly approved as good.

The diagonal line represents random guessing. Curves that bow strongly toward the top-left corner indicate better discrimination ability.

The AUC Area Under the ROC Curve; the probability that the model assigns a higher risk score to a randomly chosen positive case than to a randomly chosen negative case. summarizes the ROC curve into a single number between 0 and 1.

- 0.50 = random guessing
- 0.70–0.80 = acceptable
- 0.80–0.90 = strong
- > 0.90 = excellent discrimination

Unlike accuracy, AUC is insensitive to class imbalance and does not depend on a single threshold choice, making it useful for comparing competing classification models.

ROC vs. Precision–Recall curves
When the positive class is rare (such as loan defaults), precision–recall curves are often more informative about operational performance. ROC curves remain valuable for measuring overall ranking quality, but PR curves better reflect the cost of false positives in imbalanced datasets.

Threshold Tradeoffs and Business Costs

So far, we have evaluated models assuming a fixed classification threshold (usually 0.50). In practice, this threshold is a *business decision*, not a statistical one.

A probability threshold determines when the model predicts *default* versus *good loan*. Changing this threshold shifts the balance between:

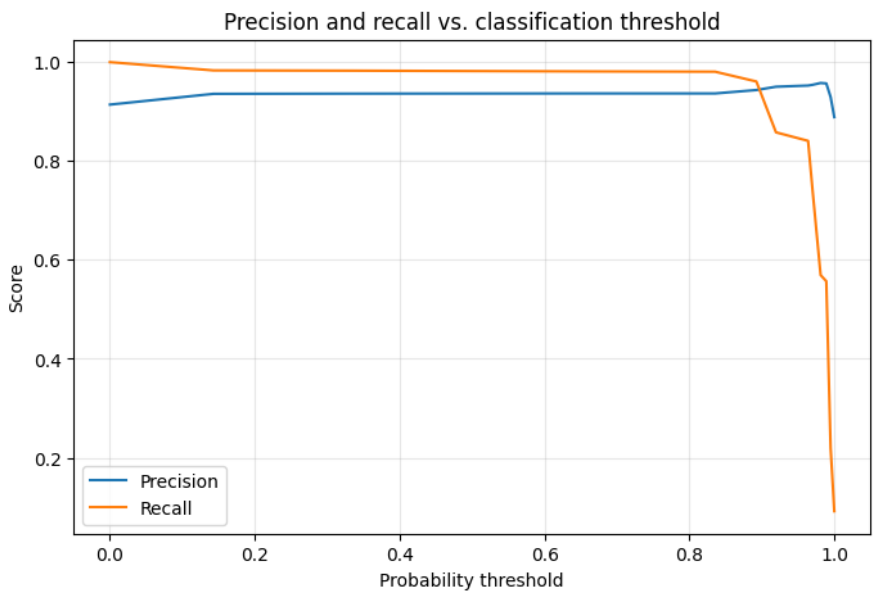
- **False positives**: approving borrowers who later default (defaults predicted as good).
- **False negatives**: rejecting creditworthy borrowers (good loans flagged as risky).

In lending, false positives (defaults approved as good) are usually the most expensive errors because they cause direct financial loss. False negatives (good borrowers rejected) represent missed revenue opportunities. This asymmetry motivates using precision-focused or probability-based metrics alongside recall.

Visualizing threshold effects

We can directly visualize how precision and recall change as the threshold moves.

```
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

y_test_prob = deep_model.predict_proba(X_test)[:, 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_test_prob)
plt.figure(figsize=(8, 5))
plt.plot(thresholds, precision[:-1], label=&quot;Precision&quot;)
plt.plot(thresholds, recall[:-1], label=&quot;Recall&quot;)
plt.xlabel(&quot;Probability threshold&quot;)
plt.ylabel(&quot;Score&quot;)
plt.title(&quot;Precision and recall vs. classification threshold&quot;)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



This plot shows that:

- Lower thresholds increase recall but reduce precision.
- Higher thresholds increase precision but reduce recall.

There is no universally optimal threshold. The correct choice depends on organizational priorities, regulatory constraints, and financial risk tolerance.

Cost-sensitive framing

Threshold selection is often formalized using a cost matrix:

Table 13.6
Example cost structure for loan default prediction

Outcome	Description	Business cost
True positive	Correctly reject risky borrower	Low
True negative	Correctly approve good borrower	Low
False positive	Reject safe borrower	Moderate (lost profit)
False negative	Approve borrower who defaults	High (financial loss)

Because false negatives are far more costly, lenders often choose thresholds that intentionally sacrifice some precision in exchange for higher recall.

Modeling judgment
Metrics guide model selection, but threshold choice reflects organizational strategy. Data scientists should present tradeoff curves and cost scenarios to stakeholders rather than selecting thresholds unilaterally.

Putting the Metrics Together

Each evaluation metric highlights a different aspect of classification performance. No single number fully describes model quality, especially when business costs are asymmetric.

The table below summarizes when each metric is most useful.

Table 13.7
Summary of classification metrics and recommended use cases

Metric	What it measures	When to prioritize it	Lending example
Accuracy	Overall correctness	Classes balanced and error costs similar	Rarely appropriate for default prediction
Precision	Reliability of positive predictions	False positives are costly	Avoid rejecting safe borrowers
Recall	Ability to detect positives	False negatives are costly	Catch most risky borrowers
F1-score	Balance of precision and recall	Need compromise between both	Moderate-risk portfolios
Log loss	Probability calibration quality	Decisions use predicted probabilities	Risk-based pricing models
ROC AUC	Ranking ability across thresholds	Comparing models independent of threshold	Early model screening

Recommended evaluation workflow

For applied classification problems, a practical workflow is:

1. Train candidate models using training data.
2. Select hyperparameters using validation log loss or ROC AUC.
3. Evaluate confusion matrix, precision, recall, and F1 on the test set.
4. Inspect probability calibration using log loss.
5. Choose a classification threshold using business cost tradeoffs.

This separation between model selection, performance evaluation, and threshold choice prevents accidental overfitting and aligns modeling decisions with business objectives.

Key takeaways

- Classification quality cannot be captured by accuracy alone.
- Probability-based metrics are essential for risk-sensitive applications.
- Thresholds encode business policy, not mathematical truth.
- Model evaluation must reflect real operational costs.

With evaluation principles established, we now turn to understanding how classification trees represent decisions internally and how to visualize their structure.

13.7Multiclass Classification

So far, we have treated classification as a two-class problem. Many real business problems, however, involve more than two meaningful outcomes. This is called **multiclass classification**: predicting which one of three or more categories a case belongs to.

In Lending Club data, the raw **loan_status** variable contains many distinct statuses. Some are common (such as *Fully Paid*), while others are rare (such as “late” variants). When categories are extremely rare, models can struggle to learn stable patterns, and evaluation becomes noisy. A common modeling approach is to *bin rare categories into broader groups* that still preserve business meaning.

Step 1: Examine loan_status frequencies

We begin by counting how frequently each raw status appears. This helps us decide which statuses are common enough to keep separate and which should be grouped.

```
# Assumes from earlier sections:
# df (DataFrame) that includes loan_status
# We will create a multiclass label based on loan_status
status_counts = df[&quot;loan_status&quot;].value_counts(dropna=False)
print(status_counts)


# Output:
# loan_status
# Current               6612
# Fully Paid            2722
# Charged Off            898
# Late (31-120 days)     152
# In Grace Period         56
# Late (16-30 days)       36
# Name: count, dtype: int64
```

In many Lending Club extracts, the “late” statuses appear relatively rarely compared to the dominant categories. If we keep every rare late category as its own class, the model may fit unstable decision rules, and metrics can be misleading because the model has too few examples to learn from.

Step 2: Create a 3-class label by binning statuses

To build a practical multiclass example, we will create three classes:

- **good**: combine *Current* and *Fully Paid*.
- **late**: bin all late-related statuses together (rare categories become one stable class).
- **bad**: combine *Default* and *Charged Off*.

This preserves the business meaning: loans are either doing fine, showing warning signs, or have clearly failed.

```
# Create a 3-class label from loan_status
# good = Current or Fully Paid
# bad = Default or Charged Off
# late = all remaining statuses (late-related and rare variants)

def recode_status_multiclass(s):
  s = str(s).strip()

  if s in [&quot;Current&quot;, &quot;Fully Paid&quot;]:
    return &quot;good&quot;

  if s in [&quot;Default&quot;, &quot;Charged Off&quot;]:
    return &quot;bad&quot;

  return &quot;late&quot;

y3 = df[&quot;loan_status&quot;].apply(recode_status_multiclass)
y3.value_counts()


# Output:
# loan_status
# good    9334
# bad      898
# late     244
# Name: count, dtype: int64
```

Always check the new class counts after recoding. If one class is still extremely small, you may need a different binning scheme, a larger dataset, or a different modeling approach.

One-vs-rest and native multiclass support

Some algorithms are naturally multiclass. For example, decision trees can directly split data to separate multiple classes in the leaves. Other algorithms, such as logistic regression, are fundamentally binary and must be extended to handle multiple classes.

A common extension method is **one-vs-rest (OvR)**. In OvR, the model trains one binary classifier per class. Each classifier learns to separate its class from “all other classes.” At prediction time, the class with the strongest predicted probability is chosen.

Step 3: Multiclass Decision Tree in Python

Decision trees in scikit-learn support multiclass classification directly. The model learns splits that reduce impurity across all classes at once.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier

# Assumes from earlier sections:
# X (features DataFrame), preprocessor (ColumnTransformer)
X_train3, X_test3, y_train3, y_test3 = train_test_split(
  X, y3, test_size=0.20, random_state=27, stratify=y3
```

```
)

tree3 = DecisionTreeClassifier(
  max_depth=3,
  min_samples_leaf=1,
  criterion=&quot;gini&quot;,
  random_state=27
)

model_tree3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;tree&quot;, tree3)
])

model_tree3.fit(X_train3, y_train3)
```

We use *stratify=y3* to preserve the class proportions in both the training and test sets. Without stratification, rare classes such as *late* may disappear entirely from one split, making evaluation unreliable.

Step 4: Multiclass logistic regression in Python

Scikit-learn logistic regression supports multiclass classification by combining multiple binary models. Under the hood, this is commonly done using a *one-vs-rest* strategy, where one classifier is trained for each class against all others. For learning purposes, we implement this strategy explicitly using *OneVsRestClassifier* so the extension from binary classification is transparent.

```
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier

lr_base = LogisticRegression(
  solver=&quot;liblinear&quot;,
  max_iter=2000,
  random_state=27
)

lr_ovr = OneVsRestClassifier(lr_base)

model_lr3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;lr&quot;, lr_ovr)
])

model_lr3.fit(X_train3, y_train3)
```

Step 5: Metrics for multiclass classification

Metrics extend naturally to multiclass settings, but there is an important choice: do we average performance across classes equally, or do we weight classes by how common they are?

Macro averaging treats each class equally, which is useful when minority classes matter. **Weighted averaging** weights each class by its frequency, which is useful when overall performance is the priority.

Baseline Accuracy in Multiclass Problems

In multiclass classification, the same “majority-class baseline” idea still applies: the baseline accuracy is the proportion of the *largest* class, because a no-skill model can predict that single class for every case.

For example, if the three classes are distributed as 90% *good*, 8% *bad*, and 2% *late*, then predicting *good* for every loan yields 90% accuracy without learning anything about risk.

This is why multiclass evaluation must go beyond accuracy: a model can post a “high” accuracy while still failing to detect the minority classes that often matter most operationally (such as *late* warning cases).

In our Lending Club test split, the *good* class is by far the largest class, so the multiclass baseline accuracy is essentially “always predict good.” The code below calculates that baseline directly from *y_test3*.

```
# Multiclass baseline accuracy = largest class proportion in the test set
base_rate = y_test3.value_counts(normalize=True).max()
majority_class = y_test3.value_counts().idxmax()

print(&quot;Majority class:&quot;, majority_class)
print(&quot;Multiclass baseline accuracy:&quot;, round(base_rate, 4))


# Output:
# Majority class: good
# Multiclass baseline accuracy: 0.8907
```

Interpretation: if a model’s accuracy is not better than 89.07% accuracy, it is not providing meaningful predictive value because it is effectively behaving like a majority-class predictor.

After checking the baseline, we evaluate models using per-class precision/recall/F1 and log loss to see whether they are actually learning the minority classes rather than defaulting to “good” predictions.

```
from sklearn.metrics import classification_report, accuracy_score, log_loss

# Predictions
y_tree3_pred = model_tree3.predict(X_test3)
y_lr3_pred = model_lr3.predict(X_test3)

print(&quot;Decision tree (3-class) accuracy:&quot;, round(accuracy_score(y_test3, y_tree3_pred), 4))
print(&quot;Logistic regression (3-class) accuracy:&quot;, round(accuracy_score(y_test3, y_lr3_pred), 4))
print(&quot;\nDecision tree (macro/weighted report):&quot;)
print(classification_report(y_test3, y_tree3_pred, digits=3, zero_division=0))
print(&quot;\nLogistic regression (macro/weighted report):&quot;)
print(classification_report(y_test3, y_lr3_pred, digits=3, zero_division=0))

# Probabilistic metric: multiclass log loss
y_tree3_prob = model_tree3.predict_proba(X_test3)
y_lr3_prob = model_lr3.predict_proba(X_test3)

print(&quot;Decision tree log loss:&quot;, round(log_loss(y_test3, y_tree3_prob), 4))
print(&quot;Logistic regression log loss:&quot;, round(log_loss(y_test3, y_lr3_prob), 4))


# Output
# Decision tree (3-class) accuracy: 0.8927
# Logistic regression (3-class) accuracy: 0.9375
#
# Decision tree (macro/weighted report):
#               precision    recall  f1-score   support
```

```
#
#          bad      0.463     0.106     0.172       180
#         good      0.901     0.992     0.944      1867
#         late      0.000     0.000     0.000        49
#
#     accuracy                          0.893      2096
#    macro avg      0.455     0.366     0.372      2096
# weighted avg      0.843     0.893     0.856      2096
#
#
# Logistic regression (macro/weighted report):
#               precision    recall  f1-score   support
#
#          bad      0.912     0.572     0.703       180
#         good      0.940     0.996     0.967      1867
#         late      0.500     0.061     0.109        49
#
#     accuracy                          0.938      2096
#    macro avg      0.784     0.543     0.593      2096
# weighted avg      0.928     0.938     0.924      2096
#
# Decision tree log loss: 0.4033
# Logistic regression log loss: 0.2246
```

At first glance, both models appear to perform well: the decision tree achieves about *89.3%* accuracy and logistic regression achieves about *93.8%* accuracy. This difference suggests that logistic regression is the stronger model overall, but accuracy alone does not tell the full story.

Because the *good* class dominates the dataset, a model can achieve high accuracy simply by predicting "good" most of the time. This is why multiclass evaluation must always go beyond accuracy to examine per-class behavior.

Notice that the decision tree reports zero precision and recall for the *late* class. This means the model never predicted this class at all. This is a common outcome when a class is rare and decision boundaries are optimized for overall accuracy rather than minority detection.

Logistic regression performs substantially better on minority classes. It achieves meaningful precision and recall for the *bad* class and nonzero recall for the *late* class.

Although recall for *late* remains low, the key improvement is that the model is at least learning a decision boundary that sometimes detects warning cases instead of ignoring them entirely.

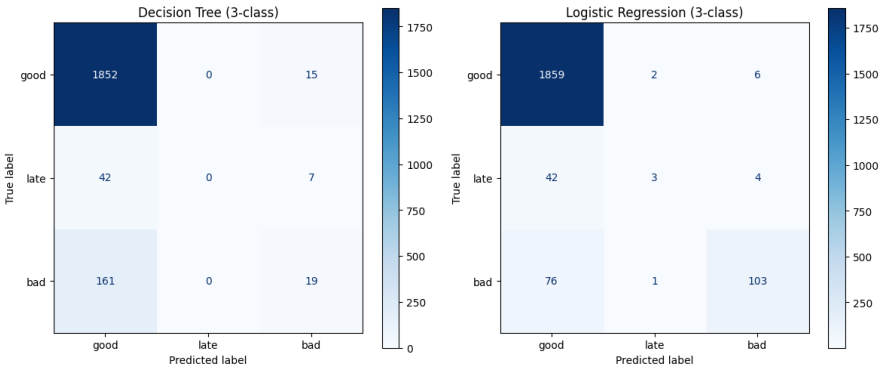
The difference in log loss is also large: approximately *0.403* for the decision tree versus *0.225* for logistic regression.

This indicates that logistic regression produces much better calibrated probability estimates across all three classes. Even when predictions are incorrect, its probability assignments tend to be less overconfident.

In risk modeling applications such as lending, probability quality is often more valuable than raw accuracy because decisions are based on risk thresholds, not just class labels.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
```

```
labels_order = [&quot;good&quot;, &quot;late&quot;, &quot;bad&quot;]
cm_tree3 = confusion_matrix(y_test3, y_tree3_pred, labels=labels_order)
cm_lr3 = confusion_matrix(y_test3, y_lr3_pred, labels=labels_order)
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
ConfusionMatrixDisplay(cm_tree3, display_labels=labels_order).plot(cmap=&quot;Blues&quot;,
values_format=&quot;d&quot;, ax=plt.gca())
plt.title(&quot;Decision Tree (3-class)&quot;)
plt.subplot(1, 2, 2)
ConfusionMatrixDisplay(cm_lr3, display_labels=labels_order).plot(cmap=&quot;Blues&quot;, values_format=&quot;d&quot;,
ax=plt.gca())
plt.title(&quot;Logistic Regression (3-class)&quot;)
plt.tight_layout()
plt.show()
```



The confusion matrices visualize these patterns directly. For the decision tree, most *late* loans are misclassified as *good*, confirming that the model does not separate warning cases into their own region of feature space.

For logistic regression, the confusion matrix shows fewer extreme failures. While some *late* cases are still confused with *good*, a portion are correctly identified, creating a usable early-risk signal.

In multiclass problems, the confusion matrix shows not only how often predictions are correct, but *which classes are being confused with each other*. Each row represents the true class, and each column represents the predicted class.

This is especially important when classes have different business meanings. In this example, confusing *late* loans with *good* loans is much more costly than confusing them with *bad* loans, because it hides early warning signs of default risk.

While accuracy and log loss summarize overall performance, the confusion matrix reveals *systematic error patterns*, such as models that completely ignore rare classes or consistently misclassify borderline cases. For operational decision-making, this class-by-class error structure is often more informative than any single summary metric.

Practical takeaway

Multiclass classification usually begins with a key modeling decision: whether categories should be kept separate or binned into larger groups. In this Lending Club example, we created three business-relevant

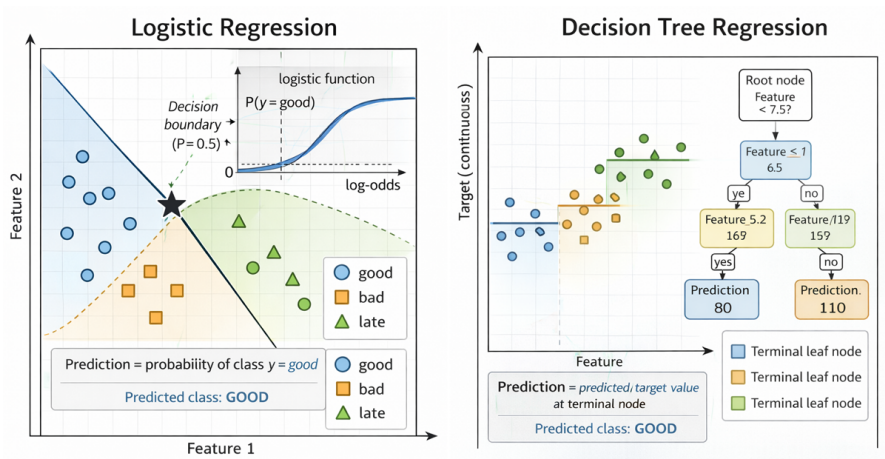
outcome classes (good, late, bad) to stabilize learning and evaluation while preserving interpretability.

Summary

Table 13.8
Binary vs Multiclass Classification

Aspect	Binary	Multiclass
Number of labels	2	3+
Logistic regression	Single model	One-vs-rest or multinomial
Decision trees	Native support	Native support
Metrics	Single confusion matrix	Per-class + averaged
Class imbalance risk	Moderate	High

13.8Logistic Regression vs Decision Trees



So far in this chapter, we have studied two core classification models: logistic regression and decision trees. Both can solve the same prediction problems, but they do so in fundamentally different ways and exhibit very different strengths and weaknesses.

Understanding these differences is more important than memorizing syntax. In practice, model selection is a design decision that balances interpretability, flexibility, stability, and risk.

Table 13.9
Logistic Regression vs Decision Trees

Aspect	Logistic Regression	Decision Tree
Interpretability	High (coefficients, odds ratios)	Medium (tree structure)
Nonlinear relationships	No (linear boundary)	Yes (automatic splits)
Feature scaling required	Yes	No
Overfitting risk	Low	High

Aspect	Logistic Regression	Decision Tree
Probability calibration	Usually good	Often poor
Deployment complexity	Simple	Simple

Logistic regression produces smooth probability estimates and stable decision boundaries. It tends to generalize well and behaves predictably when data changes slightly.

Decision trees are more flexible and can automatically capture nonlinear interactions, but this flexibility comes at a cost: they are sensitive to noise and often overfit without careful depth control.

These differences explain the multiclass results seen earlier in this chapter: the tree achieved reasonable accuracy but failed to identify rare classes, while logistic regression produced better probability estimates and more balanced class detection.

When should you use each model?

- **Choose logistic regression when:** interpretability matters, probability quality matters, the dataset is small or noisy, or regulatory transparency is required.
- **Choose decision trees when:** relationships are highly nonlinear, feature interactions are complex, or explainable rule-based decisions are useful.

In modern practice, neither model is usually the final answer. Instead, they serve as the foundation for more powerful techniques that combine many models together.

In the next chapter, we will study ensemble methods, which are specifically designed to reduce the weaknesses of individual trees while preserving their flexibility.

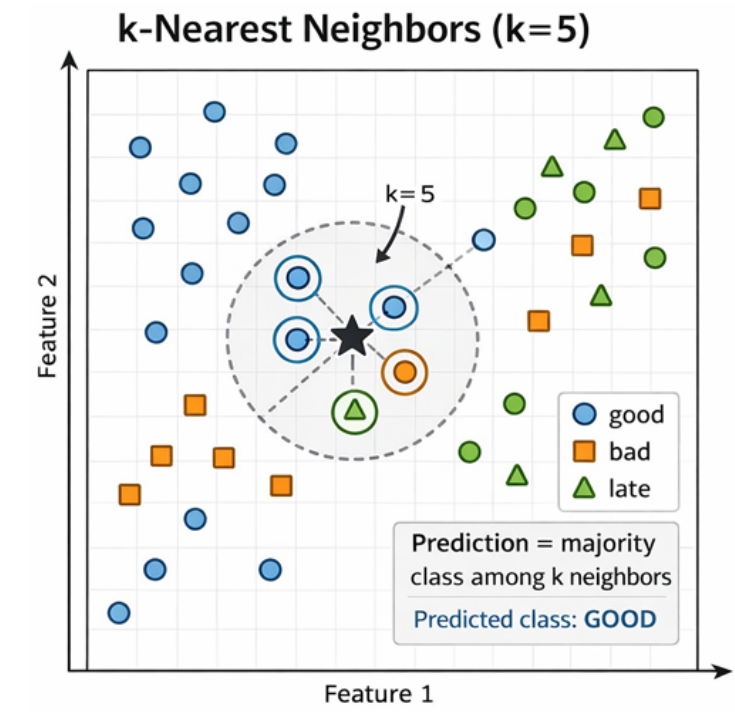
13.9Other Classification Algorithms

Logistic regression and decision trees are two of the most widely used classification models in practice. However, many other algorithms are available, each with different assumptions, strengths, and limitations.

In this section, we train several additional classifiers on the *same multiclass Lending Club problem* introduced earlier and compare their performance using the same train/test split and evaluation metrics.

This allows us to focus on how algorithm choice affects results when the data, preprocessing, and label definition remain fixed.

k-Nearest Neighbors (k-NN)



k-Nearest Neighbors classifies a new observation by finding the *k* most similar training points and assigning the majority label among them. k-NN requires no explicit training phase but is sensitive to feature scaling and becomes slow for large datasets.

This illustration to the right represents how the k-nearest neighbors (KNN) algorithm classifies a new observation based on proximity to existing labeled data points. Each colored cluster represents a different class in the training data, and the unlabeled point represents a new case whose class is unknown.

A circle drawn around the new point highlights its k closest neighbors in feature space, where distance is typically measured using Euclidean or a similar metric. The arrows from the neighboring points indicate that only these nearby observations influence the prediction, while distant points are ignored.

The predicted class is determined by a majority vote among the selected neighbors, meaning the class that appears most frequently among the k closest points is chosen. Because KNN makes no assumptions about the underlying data distribution, it can model highly irregular and nonlinear class boundaries.

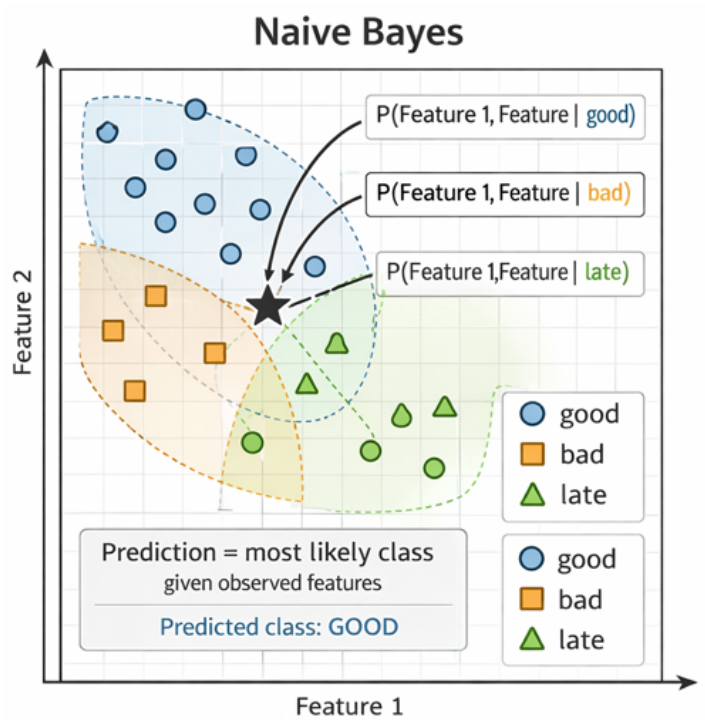
Conceptually, this image emphasizes that KNN is an example-based method that defers learning until prediction time, relying entirely on stored training data rather than an explicit mathematical model.

Next, let’s train a k-NN model in Python:

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=7)
```

```
model_knn3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;knn&quot;, knn)
])

model_knn3.fit(X_train3, y_train3)
```

Naive Bayes



Naive Bayes models use probability theory and assume that features are conditionally independent given the class label. Despite this unrealistic assumption, Naive Bayes often performs well in high-dimensional problems such as text classification.

The diagram to the right represents how the Naive Bayes classification algorithm assigns a class label using probabilities rather than geometric boundaries or distances. Each colored cluster represents an existing class in the training data, while the unlabeled point in the center represents a new observation that must be classified.

Arrows connect the new observation to probability indicators for each class, showing that Naive Bayes computes the likelihood of the observation belonging to each class separately. These probabilities are calculated by combining the conditional probabilities of each feature value given the class, under the simplifying assumption that features are independent of one another.

The model then selects the class with the highest resulting posterior probability as the predicted label. Although the independence assumption is often unrealistic in real-world data, Naive Bayes frequently

performs well in practice because it estimates probabilities efficiently and remains robust even when features are moderately correlated.

Conceptually, this image highlights that Naive Bayes classifies points by comparing probabilistic evidence across classes rather than by learning complex decision boundaries or tree structures.

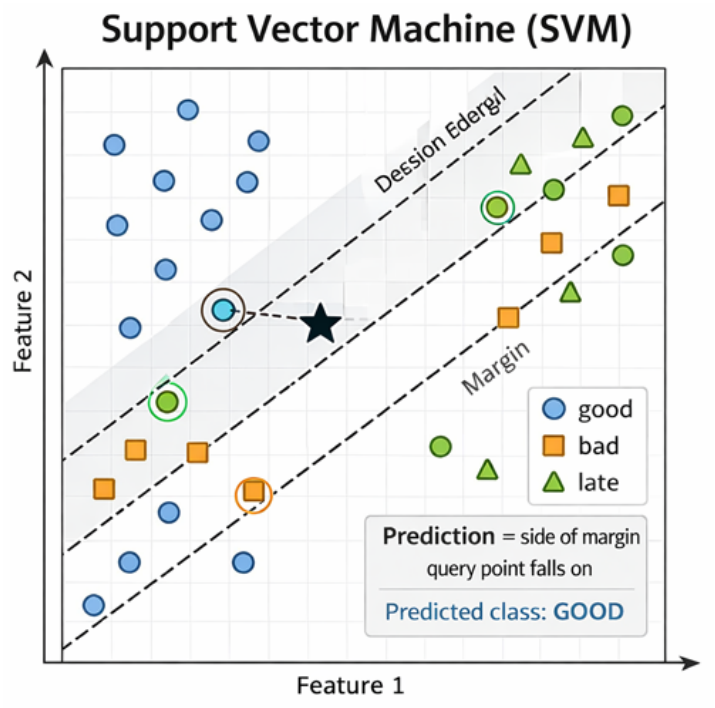
```
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import FunctionTransformer

to_dense = FunctionTransformer(lambda x: x.toarray(), accept_sparse=True)
nb = GaussianNB()

model_nb3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;to_dense&quot;, to_dense),
  (&quot;nb&quot;, nb)
])

model_nb3.fit(X_train3, y_train3)
```

Naive Bayes models in scikit-learn require *dense feature matrices*. However, modern preprocessing pipelines often produce *sparse matrices* when one-hot encoding categorical variables. To resolve this mismatch, we explicitly convert the sparse matrix to a dense array inside the pipeline before passing it to the Naive Bayes classifier.

This conversion increases memory usage and computation time, which is one reason Naive Bayes is less commonly used in high-dimensional tabular problems with many categorical variables. In contrast, logistic regression and linear SVMs are designed to operate efficiently on sparse inputs.

Practical note on sparse vs dense data
Many machine learning models accept sparse matrices directly, but some probabilistic models (including Gaussian Naive Bayes) do not. When building pipelines, always verify whether an estimator supports sparse input to avoid runtime errors and unnecessary memory usage.

Support Vector Machines (SVM)



Support Vector Machines attempt to find the decision boundary that maximizes the margin between classes. With kernel functions, SVMs can model nonlinear boundaries, but they require careful tuning and do not naturally produce well-calibrated probabilities.

This visualization to the right represents how a support vector machine (SVM) classifier separates classes by constructing an optimal decision boundary. Each colored group of points corresponds to a different class, and the straight line between them represents the learned separating hyperplane.

Two parallel margin lines are shown on either side of the decision boundary, forming a corridor that the algorithm attempts to maximize. Only a small number of points lying closest to the boundary, called support vectors, directly determine the position and orientation of this separating line.

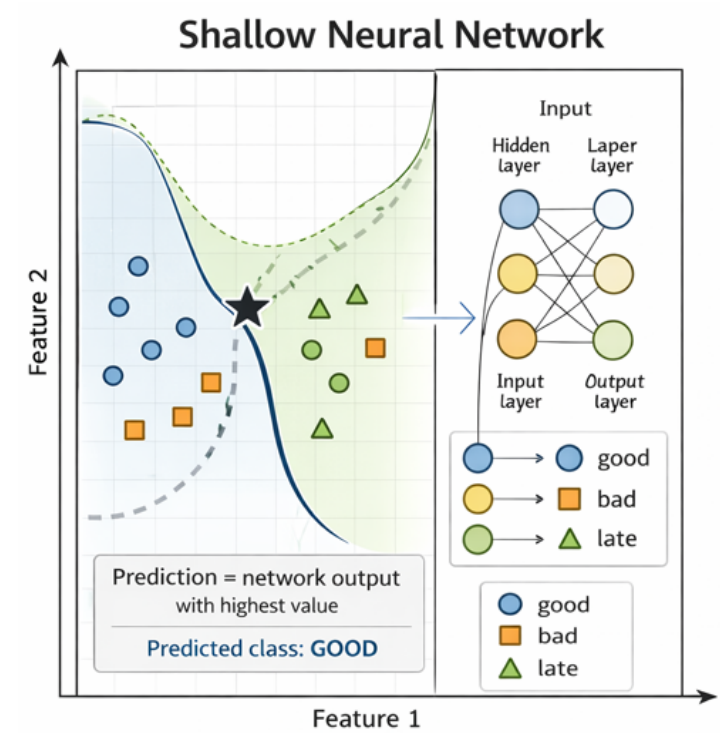
The highlighted new observation is classified according to which side of the boundary it falls on, rather than by measuring distance to many training points. By focusing on maximizing the margin, SVM aims to produce a boundary that generalizes well to unseen data and is robust to small perturbations.

Conceptually, this image shows that SVM classification depends on geometric separation and a small set of critical training points, rather than overall class averages or probability estimates.

```
from sklearn.svm import SVC

svm = SVC(kernel=&quot;rbf&quot;, probability=True, random_state=27)

model_svm3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;svm&quot;, svm)
```

```
])
model_svm3.fit(X_train3, y_train3)
```

Shallow Neural Network (MLP)



Neural networks learn nonlinear transformations of the input features using layers of weighted combinations. Here we use a small multilayer perceptron (MLP) to demonstrate the basic idea without entering deep learning territory.

The image to the right illustrates the structure and logic of a **shallow neural network** used for multiclass classification. On the left, colored input nodes represent standardized feature values from a single loan record, such as income, credit history, or loan amount. These inputs are passed forward through weighted connections into a single hidden layer of neurons shown in the center. This illustration is simplified for clarity and captures the main architectural idea.

Each hidden neuron computes a weighted sum of its inputs and applies a nonlinear activation function, allowing the model to capture interactions and curved decision boundaries that linear models cannot represent. The arrows emphasize that information flows in one direction only, from inputs to hidden layer to outputs, which is why this architecture is called a *feedforward* network.

On the right, three output nodes correspond to the multiclass targets used in this chapter: *good*, *late*, and *bad*. The values at these nodes represent predicted probabilities after a softmax transformation, and the class with the highest probability becomes the final prediction.

The diagram highlights why shallow neural networks are more flexible than logistic regression but still simpler than deep learning models. With only one hidden layer, they can model moderate nonlinearity while remaining relatively fast to train and easier to tune. In later chapters, deeper neural networks extend this same structure by stacking many hidden layers to learn more complex representations.

```
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(
  hidden_layer_sizes=(32, 16),
  max_iter=500,
  random_state=27
)

model_mlp3 = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;mlp&quot;, mlp)
])

model_mlp3.fit(X_train3, y_train3)
```

Performance comparison

We now evaluate all models on the same test set using accuracy and multiclass log loss.

```
import pandas as pd
from sklearn.metrics import accuracy_score, log_loss

models = {
  &quot;Decision Tree&quot;: model_tree3,
  &quot;Logistic Regression&quot;: model_lr3,
  &quot;k-NN&quot;: model_knn3,
  &quot;Naive Bayes&quot;: model_nb3,
  &quot;SVM&quot;: model_svm3,
  &quot;Neural Network&quot;: model_mlp3
}

results = []

for name, model in models.items():
  y_pred = model.predict(X_test3)
  y_prob = model.predict_proba(X_test3)

  results.append({
    &quot;model&quot;: name,
    &quot;accuracy&quot;: accuracy_score(y_test3, y_pred),
    &quot;log_loss&quot;: log_loss(y_test3, y_prob)
  })

results_df = pd.DataFrame(results).sort_values(&quot;accuracy&quot;, ascending=False)
results_df
```

	model	accuracy	log_loss
1	Logistic Regression	0.937500	0.224604
5	Neural Network	0.923187	0.422983
4	SVM	0.904103	0.204720
0	Decision Tree	0.892653	0.403342
2	k-NN	0.890267	1.814206
3	Naive Bayes	0.378340	22.406909

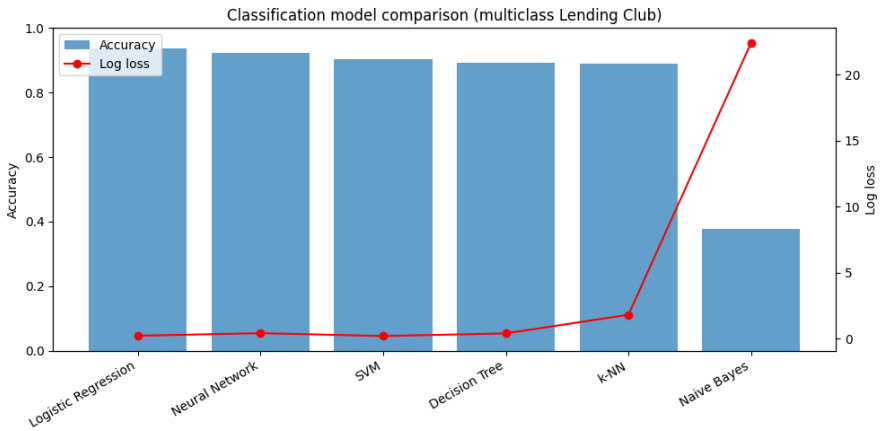
In most business datasets, logistic regression and shallow neural networks often provide the best probability quality, while trees and k-NN tend to struggle with minority classes.

SVMs can perform well but are harder to tune and interpret. Naive Bayes is usually fastest but rarely the most accurate. Let's put together a visual comparison of the results.

Visual comparison of model performance

The chart below compares all models using both accuracy (higher is better) and log loss (lower is better). Viewing both metrics together highlights the difference between correct classification and probability quality.

```
import matplotlib.pyplot as plt

plot_df = results_df.copy()
fig, ax1 = plt.subplots(figsize=(10, 5))

# Accuracy bars (left axis)
ax1.bar(plot_df[&quot;model&quot;], plot_df[&quot;accuracy&quot;], alpha=0.7, label=&quot;Accuracy&quot;)
ax1.set_ylabel(&quot;Accuracy&quot;)
ax1.set_ylim(0, 1)

# Log loss line (right axis)
ax2 = ax1.twinx()
ax2.plot(plot_df[&quot;model&quot;], plot_df[&quot;log_loss&quot;], marker=&quot;o&quot;, color=&quot;red&quot;,
label=&quot;Log loss&quot;)
ax2.set_ylabel(&quot;Log loss&quot;)

# Title and x-axis formatting
ax1.set_title(&quot;Classification model comparison (multiclass Lending Club)&quot;)
plt.setp(ax1.get_xticklabels(), rotation=30, ha=&quot;right&quot;)

# Combined legend
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc=&quot;upper left&quot;)
fig.tight_layout()
plt.show()
```



Why not ensembles (yet)?

Many of the weaknesses observed above—tree instability, poor minority detection, and probability miscalibration—are precisely what *ensemble algorithms* are designed to fix.

Random forests, gradient boosting, and boosting-based methods combine many weak models to produce dramatically more stable and accurate classifiers. Because ensembles represent a major conceptual shift, they are covered in their own dedicated chapter.

Why not deep neural networks?

Deep neural networks require different training procedures, regularization strategies, and hardware considerations. They are extremely powerful but demand more data, tuning, and theoretical background than is appropriate at this stage. For tabular business data, simpler models often outperform deep networks when properly tuned.

Practical takeaway

Logistic regression remains the best default for interpretable probability-based classification. Decision trees offer transparency and flexibility but require careful regularization. Other algorithms are valuable tools, but ensembles usually dominate modern applied classification. But now that you understand the basics of classification modeling, you'll be well prepared to shift to ensemble methods in a later chapter.

Model comparison summary

Table 13.10
Comparison of classification algorithms on the Lending Club multiclass problem

Model	Accuracy	Log loss	Interpretability	Handles nonlinearity	Probability quality	Overfitting risk

Model	Accuracy	Log loss	Interpretability	Handles nonlinearity	Probability quality	Overfitting risk
Logistic Regression	High	Low (best)	High	No	Excellent	Low
Decision Tree	Moderate	Moderate	Medium	Yes	Often poor	High
k-NN	Moderate	Moderate	Low	Yes (implicit)	Poor	Medium
Naive Bayes	Low–Moderate	High	Medium	No	Poor	Low
SVM (RBF)	High	Moderate	Low	Yes	Moderate	Medium
Neural Network (MLP)	High	Low	Low	Yes	Good	Medium

13.10Case Studies

See how well you understand the chapter concepts by working through the practice problems below:

Case #1: Customer Churn Dataset
This practice uses a **Customer Churn** dataset. Your goal is to build **predictive classification models** that estimate whether a customer will *churn* (leave the service). You will train and compare a *logistic regression* model and a *decision tree*, evaluate their performance using classification metrics, and examine how tree depth affects overfitting.

Dataset attribution: This dataset is a commonly used telecommunications churn dataset containing customer demographics, service usage, contract details, and a binary churn outcome variable. [See details on Kaggle.com]

This download can be found online.

Prediction goal: Predict the binary outcome *Churn* (Yes / No) using all available customer attributes except the target variable itself. You should treat this as a supervised classification problem using a full machine learning workflow with preprocessing pipelines.

For reproducibility, use **random_state = 27** everywhere that a random seed is accepted

Tasks

- Inspect the dataset: number of rows and columns, data types, missing values, and the class distribution of *Churn*.
- Create X (features) and y (target), where y = *Churn*. Encode the target as binary (0/1 or No/Yes).

- Split the data into *training* and *test* sets (80/20) using *random_state=27* and *stratify=y*.
- Build an sklearn preprocessing pipeline using a *ColumnTransformer*: scale numeric features with *StandardScaler* and one-hot encode categorical features using *OneHotEncoder(handle_unknown="ignore")*.
- Establish a baseline classifier that always predicts the most common class. Report baseline test-set *accuracy* and *log loss*.
- Train a *logistic regression* classifier inside the pipeline. Report test-set *accuracy*, *log loss*, and a classification report (precision, recall, F1 by class).
- Train a *DecisionTreeClassifier* with *max_depth=3* inside the same preprocessing pipeline. Report the same evaluation metrics.
- Plot and compare the *confusion matrices* for both models. Comment on the types of errors each model makes.
- Visualize the trained decision tree using *sklearn.tree.plot_tree* with *max_depth = 3* for readability.
- Extract and plot impurity-based *feature importances* from the trained tree. Report the top three features.
- Train at least three additional decision trees using different values of *max_depth* (for example: 2, 5, and unrestricted). Record training and test-set accuracy for each.
- Create a small table or plot showing how training accuracy and test accuracy change as tree depth increases.

Analytical questions

1. How many rows and columns are in the Customer Churn dataset?
2. What percentage of customers in the dataset have churned?
3. What are the baseline model's test-set *accuracy* and *log loss*? Briefly explain why log loss is so poor for a majority-class baseline.
4. What are the logistic regression model's test-set *accuracy* and *log loss*? Include a classification report and interpret precision vs recall for the churn class.
5. What are the decision tree model's test-set *accuracy* and *log loss* at *max_depth=3*? Include a classification report and compare it to logistic regression.
6. Which model performed better overall? Which model had higher recall for the churn class?
7. Include both confusion matrices. Which model produces more false negatives for churn, and why might that matter to a retention team?
8. Extract and report the top three impurity-based feature importances from the decision tree. Which features appear most influential for predicting churn?
9. From your depth sweep, which *max_depth* is best by test *log loss*, and which is best by test *accuracy*?

10. At what depth did the tree begin to show signs of overfitting? Explain using training vs test accuracy or log loss.
11. Short reflection (3–5 sentences): Why might logistic regression outperform a deep decision tree on this dataset? How does this relate to bias–variance tradeoff and model interpretability?

Customer Churn Classification Practice Answers

These answers use the Customer Churn dataset with an 80/20 stratified train/test split (*random_state=27*) and evaluation on the holdout test set. Exact values are from the reference solution; your results should be very close if you use the same random seed and preprocessing pipeline.

Q1. Dataset size

The dataset contains **7032 rows** and **21 columns**.

Q2. Churn rate

The churn rate (share of customers labeled *Yes*) is **0.2658** (about **26.58%**).

Q3. Baseline model (majority class)

The baseline model predicts the most common class (*No*) for every customer. This yields a test-set **accuracy of 0.7342**.

The baseline test-set **log loss is 9.5809**, which is extremely poor because the baseline assigns near-certain probability to the wrong class for all churners (highly overconfident mistakes are heavily penalized by log loss).

Q4. Logistic regression results

The logistic regression model achieves test-set **accuracy = 0.8038** and **log loss = 0.4275**, substantially improving both classification accuracy and probability quality compared to the baseline.

From the classification report, logistic regression performs strongly on the majority class (*No*) with *precision = 0.851* and *recall = 0.888*. For the churn class (*Yes*), performance is weaker with *precision = 0.648* and *recall = 0.572*, meaning the model misses a meaningful portion of churners at the default threshold.

Macro averages (*macro avg f1 = 0.739*) summarize balanced performance across classes, while weighted averages (*weighted avg f1 = 0.800*) reflect the dominance of the non-churn class in the dataset.

Q5. Decision tree (max_depth = 3) results

The depth-3 decision tree achieves test-set **accuracy = 0.7783** and **log loss = 0.4505**. This is better than baseline but worse than logistic regression on both metrics in this run.

The decision tree shows a strong tendency to predict *No*. In the classification report, the churn class (*Yes*) has *recall = 0.350*, meaning it finds only about 35% of actual churners. This is consistent with the confusion matrix showing many churners misclassified as non-churners at the default threshold.

Q6. Model comparison

Overall, logistic regression outperformed the depth-3 decision tree on both accuracy (0.8038 vs 0.7783) and log loss (0.4275 vs 0.4505) in this run.

Logistic regression also had substantially higher recall for the churn class (0.572 vs 0.350). If the business goal is to proactively intervene with likely churners, this recall gap matters more than overall accuracy. However, one model may be more conservative (fewer false alarms) while another is more sensitive (catches more positives). The “best” choice depends on the business costs of false positives vs false negatives.

Q7. Confusion matrices

Logistic regression confusion matrix (test set): **TN = 917**, **FP = 116**, **FN = 160**, **TP = 214**. The model correctly identifies 214 churners but misses 160 churners.

Decision tree (depth = 3) confusion matrix (test set): **TN = 964**, **FP = 69**, **FN = 243**, **TP = 131**. The tree produces fewer false positives than logistic regression, but it misses far more churners (243 vs 160), which drives its low recall for the churn class. For a retention team, these extra false negatives mean 83 additional churners go undetected and receive no intervention.

Q8. Feature importances

The three most important features (impurity-based importance) in the tree are:

1. **Contract_Month-to-month** (importance *0.612992*)
2. **InternetService_Fiber optic** (importance *0.177452*)
3. **TotalCharges** (importance *0.114434*)

Interpretation: month-to-month contracts dominate the tree’s early splits, suggesting contract type is highly informative for churn risk in this dataset. Fiber optic internet service and billing-related measures (total charges, tenure, monthly charges) also contribute, but to a smaller extent in the learned tree.

Q9. Best depth by metric

The “best” depth depends on the metric:

- **Best by test log loss: max_depth = 4** (test log loss = *0.434553*).
- **Best by test accuracy: max_depth = 6** (test accuracy = *0.790334*).

A practical modeling judgment is to prefer the depth that optimizes *log loss* when predicted probabilities drive decisions (rankings, thresholds, expected-cost calculations). If the operational workflow only uses hard class predictions, maximizing accuracy may be acceptable, but it can hide poorly calibrated probabilities.

Q10. Overfitting onset

The depth sweep shows that training performance improves monotonically as depth increases, but test performance improves only up to a point and then degrades. This is classic overfitting: deeper trees fit noise and become overconfident on the training data, which increases test log loss and reduces test accuracy after the best region.

Test log loss becomes very large at deeper depths (for example, depth 10+), which strongly indicates the model is producing extremely confident wrong probabilities on the test set. That is why log loss is often a more sensitive diagnostic of overfitting than accuracy.

Q11. Short reflection

Deeper trees can keep reducing training error because they can create very specific splits that memorize patterns in the training data, including noise. However, those highly specific rules often do not generalize, so test performance stops improving and may worsen. This is the bias–variance tradeoff: shallow trees have higher bias (underfit), while deep trees have higher variance (overfit). Validation or test curves help identify the depth range where generalization is best.

Case #2: Employee Attrition Dataset
This practice uses the **Employee Attrition** dataset (provided as *Employee_Attrition.csv*). Your goal is to build a **predictive** classification model that estimates whether an employee will leave the company (*Attrition* = Yes/No). You will compare a **logistic regression** model and a **decision tree** model, evaluate them using classification metrics (accuracy, precision, recall, F1) and probabilistic metrics (log loss, ROC/AUC), and explore how tree depth affects overfitting.

Dataset attribution: This dataset is widely distributed as an “Employee Attrition / HR Analytics” teaching dataset (often based on IBM HR sample data). Your version is provided in this course as *Employee_Attrition.csv*. See details on Kaggle.com

This download can be found online.

Prediction goal: Predict whether *Attrition* is Yes (employee leaves) or No (employee stays) using the other columns as predictors. Use a predictive workflow with a stratified train/test split, preprocessing in an sklearn pipeline, and evaluation on a holdout test set.

For reproducibility, use **random_state = 27** everywhere that a random seed is accepted

Tasks

- Inspect the dataset: number of rows/columns, data types, missing values, and the distribution of the *Attrition* label.
- Define *X* and *y* where *y = Attrition* and *X* contains all remaining predictors. Remove ID-like columns (for example *EmployeeNumber*) if present.
- Split the data into *training* and *test* sets (80/20) using *random_state=27* and *stratify=y*.
- Build an sklearn preprocessing pipeline: scale numeric predictors (*StandardScaler*) and one-hot encode categorical predictors (*OneHotEncoder* with *handle_unknown="ignore"*). Fit preprocessing only on the training data.
- Establish a baseline classifier that always predicts the most common class. Report baseline test-set *accuracy* and baseline *log loss*.
- Train a *LogisticRegression* classifier inside your pipeline. Evaluate test-set accuracy, log loss, and a classification report (precision, recall, F1 by class).
- Train a *DecisionTreeClassifier* (start with *max_depth=3*) inside your pipeline. Evaluate test-set accuracy, log loss, and a classification report.
- Create confusion matrices for both models and interpret what types of errors each model makes (false positives vs false negatives).
- Perform a depth sweep for the decision tree (for example: 1, 2, 3, 4, 5, 6, 8, 10, 12, 15). Create two validation-curve plots: accuracy vs depth and log loss vs depth (train vs test).
- Choose a “best” tree depth using test performance. Report the best depth according to test *log loss* and the best depth according to test *accuracy*, and explain why these choices might differ.

Analytical questions

1. How many rows and columns are in the Employee Attrition dataset?
2. What proportion of employees have *Attrition = Yes*?
3. What are the baseline model’s test-set *accuracy* and *log loss*? What does the baseline log loss tell you about the probability quality of always predicting the majority class?
4. What are the logistic regression model’s test-set *accuracy* and *log loss*? Include a classification report and interpret precision vs recall for the *Attrition = Yes* class.

5. What are the decision tree model's test-set *accuracy* and *log loss* at *max_depth=3*? Include a classification report and compare it to logistic regression.
6. Include both confusion matrices. Which model produces more false negatives for attrition, and why might that matter to an HR team?
7. Extract and report the impurity-based feature importances from the depth-3 decision tree (all features with nonzero importance). Which features appear most influential?
8. From your depth sweep, which *max_depth* is best by test *log loss*, and which is best by test *accuracy*? Provide the depth-sweep table and the two validation-curve plots.
9. Short reflection (3–5 sentences): Explain how the bias–variance tradeoff appears in your depth-sweep plots (training vs test curves). Identify where overfitting begins and support your claim using accuracy and/or log loss patterns.

Employee Attrition Classification Practice Answers

These answers assume you used the Employee Attrition dataset provided, treated *Attrition* as the binary label (Yes/No), used an 80/20 train/test split, and evaluated models using accuracy, log loss, and the classification report. Your specific numeric results may differ slightly if your preprocessing, random seed, or split differs.

1. **Q1 (rows, columns):** The dataset contains *1470* rows and *35* columns.
2. **Q2 (attrition rate):** The proportion of employees with *Attrition = Yes* is *0.1612* (about *16.12%*). This indicates a moderately imbalanced classification problem where the *No* class is much more common.
3. **Q3 (baseline model):** Predicting the most common class (*No*) for every case yields *baseline accuracy = 0.8401*. The *baseline log loss = 0.4395* reflects the fact that a hard, always-majority prediction provides poor probability estimates (and would be even worse if probabilities were treated as 0/1 with no smoothing).
4. **Q4 (logistic regression):** Logistic regression achieved *accuracy = 0.8878* and *log loss = 0.3511*, improving on the baseline in both overall correctness and probability quality. In the classification report, the *Yes* class shows *precision = 0.719* and *recall = 0.489*, meaning the model identifies about half of true attrition cases while keeping false alarms relatively contained. The *No* class remains very strong (*recall = 0.964*), which is common in imbalanced settings.
5. **Q5 (decision tree, max_depth = 3):** The depth-3 decision tree achieved *accuracy = 0.8435* and *log loss = 0.3899*. While accuracy is close to the baseline, the key weakness is minority-class detection: the *Yes* class has *recall = 0.213*, meaning the tree catches only about 21% of actual attrition cases. This usually happens because a shallow tree tends to favor the majority class unless splits strongly isolate the minority class early.

6. **Q6 (confusion matrix interpretation):** From the confusion matrices, logistic regression produces more true positives (correct *Yes* predictions) than the tree, at the cost of some additional false positives. The decision tree (depth=3) misses many more attrition cases (false negatives), which aligns with its low *Yes* recall.
7. **Q7 (top feature importances):** The most influential predictors in the depth-3 tree (by impurity-based importance) were: *OverTime_No* (0.299), *MonthlyIncome* (0.264), *TotalWorkingYears* (0.103), *HourlyRate* (0.102), *JobRole_Sales Executive* (0.101), and *Age* (0.095). These suggest the tree is using workload and compensation/career-stage signals to separate lower-risk vs higher-risk groups. Note that impurity importances can over-emphasize features with many possible split points and should be interpreted as a rough heuristic rather than a causal ranking.
8. **Q8 (depth sweep and model selection):** The depth sweep shows classic overfitting: training accuracy rises steadily with depth (up to 0.9966), while test accuracy peaks early and then fluctuates; test log loss worsens rapidly for deeper trees (e.g., > 3 at depth 8 and > 5 by depth 12–15). Using the test-set criteria reported, the best depth is *max_depth = 2* for both *test log loss* and *test accuracy*. This indicates that a very shallow tree generalizes best on this dataset and that deeper trees become increasingly overconfident on wrong predictions.
9. **Q9 (reflection guidance):** A deeper tree can always reduce training error because it can keep splitting until it memorizes patterns (including noise). However, those extra splits often fit idiosyncrasies of the training set that do not repeat in new data, so test performance can stagnate or worsen. This is the bias–variance tradeoff: shallow trees have higher bias (underfitting) while deep trees have higher variance (overfitting). The log-loss curve is especially sensitive here because overfit trees often output extreme probabilities that are heavily penalized when incorrect.

Overall takeaway: Logistic regression performed best in this run because it improved both accuracy and probability quality (log loss) while achieving substantially better detection of the minority *Yes* class than the shallow tree. The depth sweep reinforces that, for trees, controlling complexity is essential—small increases in depth can quickly degrade probability calibration and generalization.

Case #3: Telco Support Ticket Priority Dataset

This practice uses a Telco customer support dataset of service tickets. Your goal is to build a **multiclass classification** model that predicts ticket *priority* (Low, Medium, High) using structured ticket and customer context variables. You will compare multiple classification algorithms from this chapter and evaluate them using both *accuracy* (threshold-based) and *multiclass log loss* (probability-based).

Dataset attribution: The dataset file for this practice is *Support_tickets.csv*. See details on Kaggle.com

This download can be found online.

Prediction goal: Predict *priority* (Low, Medium, High). In this practice, treat the classes as *nominal* (not ordered) and use standard multiclass classification algorithms and metrics.

Recommended feature set: Use a mix of numeric and categorical predictors. For clarity, use the human-readable categorical columns (for example: *day_of_week*, *company_size*, *industry*, *customer_tier*, *region*, *product_area*, *booking_channel*, *reported_by_role*, *customer_sentiment*) and numeric operational columns (for example: *org_users*, *past_30d_tickets*, *past_90d_incidents*, *customers_affected*, *error_rate_pct*, *downtime_min*, plus the binary flags such as *payment_impact_flag* and *security_incident_flag*). Exclude identifier-like columns such as *ticket_id*.

Note on duplicate encodings: This dataset includes both readable categorical columns (for example, *industry*) and numeric coded versions (for example, *industry_cat*). For this practice, choose *one approach* and be consistent. The recommended approach is to use the readable categorical columns with one-hot encoding and ignore the *_cat* versions.

For reproducibility, use **random_state = 27** everywhere that a random seed is accepted (for example: *train_test_split*, *LogisticRegression*, *DecisionTreeClassifier*, and *MLPClassifier*).

Tasks

- Inspect the dataset: report the number of rows and columns, confirm the target label values in *priority*, and compute the class proportions (counts and percentages).
- Create *X* and *y* where *y = priority*. Exclude *ticket_id* and exclude all *_cat* columns (use the readable categories instead).
- Split the data into *training* and *test* sets (80/20) using *random_state=27* and *stratify=y*.
- Build a preprocessing pipeline with *StandardScaler* for numeric predictors and *OneHotEncoder(handle_unknown="ignore")* for categorical predictors. For this practice, configure the encoder to return dense output (*sparse_output=False*) so the same preprocessing works across all algorithms.
- Establish a baseline classifier: predict the *most frequent class* for all test cases. Report baseline *accuracy*. Then compute baseline *multiclass log loss* by predicting the *training-set class proportions* as constant probabilities for every test case.
- Train and evaluate a multiclass *logistic regression* model using a one-vs-rest strategy (via *OneVsRestClassifier*). Report test-set *accuracy*, *classification report* (macro and weighted), and *log loss*.
- Train and evaluate a multiclass *decision tree classifier* (start with *max_depth=3*). Report the same metrics as in Task 6. Then run a small depth sweep (for example: 1, 2, 3, 4, 5, 6, 8, 10, 12) and identify the best depth by test *log loss*.
- Extract and plot the tree's impurity-based *feature importances*. Create a top-15 table of importance values. (If you one-hot encoded categorical variables, you may report the one-hot feature names directly.)
- Train and evaluate the additional classification algorithms from Section 13.9 as multiclass models: *k-NN*, *Naive Bayes*, *SVM*, and a *shallow neural network* (MLP). Use the same train/test split and the

same preprocessing pipeline for fair comparison. Bear in mind, shallow neural networks can take longer to train than the other models in this practice. If your MLP model is slow, apply the time-saving steps below (in order) until training completes quickly.

- **Use early stopping:** Set *early_stopping=True* so training stops when validation performance stops improving.
- **Reduce the maximum iterations:** Start with *max_iter=300* (or *500*). Increase only if the model does not converge.
- **Use a smaller hidden layer:** Start with *hidden_layer_sizes=(15,)* instead of larger sizes.
- **Limit the dataset size for experimentation:** If needed, train the MLP on a random subset of the training data (for example, 3000 rows) using a fixed seed of *27*, then rerun on the full training set once your code works.
- For example:

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline

# Time-efficient MLP settings (start here)
mlp = MLPClassifier(
  hidden_layer_sizes=(15,),
  activation=&quot;relu&quot;,
  solver=&quot;adam&quot;,
  alpha=0.0005,
  batch_size=256,
  learning_rate_init=0.001,
  early_stopping=True,
  validation_fraction=0.10,
  n_iter_no_change=10,
  max_iter=300,
  random_state=SEED
)

model_mlp = Pipeline(steps=[
  (&quot;prep&quot;, preprocessor),
  (&quot;float&quot;, EnsureFloat64()),
  (&quot;mlp&quot;, mlp)
])
```

- Create a single comparison table that lists all models (baseline, logistic regression, decision tree, k-NN, Naive Bayes, SVM, shallow neural network) and includes *test accuracy* and *test log loss*.
- Create a visual comparison chart. Because accuracy and log loss are on different scales, use a two-axis chart (left axis for accuracy bars, right axis for log loss line) so one extreme log loss value does not flatten the accuracy bars.
- Create a multiclass *confusion matrix* for your two best-performing models (choose based on log loss and macro averages). Briefly interpret which classes are most frequently confused.
- Write a short model selection checklist (5–8 bullets) describing how you would choose among these models in a real support operation, considering interpretability, probability quality, minority-class performance, and deployment constraints.

Model training reminders

- For multiclass logistic regression, use *OneVsRestClassifier(LogisticRegression(...))* to avoid deprecated multiclass settings and to keep the one-vs-rest idea explicit.

- For SVM, enable probability estimates (*probability=True*) if you want log loss; this increases training time but allows *predict_proba*.
- For Naive Bayes, using dense features is acceptable in this dataset because the one-hot expansion is manageable; that is why the encoder is configured with *sparse_output=False*.
- For the shallow neural network, use *MLPClassifier* with one hidden layer and early stopping; do not attempt deep architectures here.
- **Ensembles are intentionally excluded:** Random forests, gradient boosting, and stacking typically outperform single models but deserve their own dedicated chapter because they introduce new concepts (bagging, boosting, model aggregation, and tuning strategies).

Analytical questions

1. How many rows and columns are in the Support Tickets dataset?
2. What are the counts and percentages of each *priority* class?
3. What are the baseline model's test-set *accuracy* and *log loss*?
4. What are the logistic regression model's test-set *accuracy* and *log loss*? Which class has the lowest recall, and why might that be operationally important?
5. What are the decision tree model's test-set *accuracy* and *log loss* at *max_depth=3*?
6. In the depth sweep, which max depth performed best by test *log loss*? Did that same depth also maximize test accuracy?
7. Which 5–10 features (or one-hot encoded feature names) appear most important for the tree? Provide a brief business interpretation of the top two.
8. Compare the multiclass confusion matrices for your two best models. Which priority levels are most commonly confused with each other?
9. Fill in your model comparison table for all models. Which model has the best *log loss*? Which model has the best *macro average F1*?
10. Short reflection (5–8 sentences): If you were implementing this in a real support operation, which model would you choose and why? Address interpretability, probability quality, and the cost of confusing High priority with Low or Medium.

Support Tickets Multiclass Practice Answers

These answers are based on the Support Tickets dataset results you reported (50,000 rows) and the multiclass priority label with three unordered classes: *low*, *medium*, and *high*.

1. The Support Tickets dataset contains *50,000* rows and *33* columns.

2. Priority class distribution: *low = 25,000 (50.0%)*, *medium = 17,500 (35.0%)*, and *high = 7,500 (15.0%)*. This is an imbalanced multiclass label because the majority class (*low*) is much more common than *high*.
3. (Baseline) The baseline model achieved *test accuracy = 0.5000* and *test log loss = 0.9986*. The baseline accuracy matches the majority class rate (50%), which means it is essentially behaving like a "predict low for everything" strategy.
4. (Logistic regression OvR) The logistic regression model achieved *test accuracy = 0.8536* and *test log loss = 0.4298*. The class with the lowest recall is *medium* (*recall = 0.753*, slightly lower than *high* at 0.770). Operationally, low recall for *medium* means many true medium-priority tickets are being downgraded to *low* (or occasionally escalated), which can distort staffing forecasts and response-time commitments for the "middle tier" workload.
5. (Decision tree, max_depth = 3) The depth-3 decision tree achieved *test accuracy = 0.7752* and *test log loss = 0.5239*. Compared to logistic regression, this tree is less accurate overall and produces weaker probability estimates (higher log loss).
6. (Depth sweep) The best max depth by *test log loss* was *6* (test log loss = *0.3476*). The depth that maximized *test accuracy* was *12* (test accuracy = *0.9195*). These are not the same depth, which illustrates a common pattern: deeper trees can improve threshold-based accuracy while harming probability quality (log loss rises sharply at depths 10–12).
7. (Top feature importances, tree) The most important one-hot or raw features reported were: *customers_affected*, *downtime_min*, *error_rate_pct*, *customer_tier_Enterprise*, *customer_tier_Plus*, *reported_by_role_c_level*, *payment_impact_flag*, and smaller contributions from *product_area_analytics*. Business interpretation: (1) *customers_affected* dominates because impact scope is a direct driver of urgency—issues affecting many customers are more likely to be classified as *high*. (2) *downtime_min* is also a major driver because longer outages typically imply larger revenue, SLA, and reputational risk, which justifies higher priority.
8. (Confusion matrices) The two strongest overall models are *Shallow NN (MLP)* and *SVM (RBF)*. In multiclass triage settings, the most common confusion typically occurs between neighboring operational categories: *high vs medium* and *medium vs low*. In both the MLP and SVM confusion matrices, the off-diagonal errors concentrate between adjacent priority levels rather than at the extremes (*high* ↔ *low* swaps are rare). The *high* class tends to have the lowest per-class recall because it is the smallest class (15%), so both models occasionally misclassify true *high* tickets as *medium*. The MLP typically produces fewer of these critical misclassifications than SVM, which is one reason it leads on macro-average F1.
9. (Model comparison) Your completed table shows the best log loss is from *Shallow NN (MLP)* with *log loss = 0.1393*. The best macro-average F1 among the models that reported it is also the *Shallow NN (MLP)* (macro avg F1 = *0.942*). The SVM's macro F1 is also high (0.899); because *probability=True* was set, log loss can be computed for the SVM as well, though SVM probabilities are estimated via Platt scaling and may be less well-calibrated than those from logistic regression or the neural network.

10. Reflection example (one strong answer): In a real support operation, I would likely choose the *Shallow NN (MLP)* as the production model because it delivers the strongest overall performance on both dimensions that matter: it has the best accuracy (*0.9446*) and the best probability quality (log loss *0.1393*). Probability quality matters because triage often uses risk thresholds to route tickets into queues (auto-escalate, human review, standard queue), not just a single hard label. However, I would pair the MLP with a simpler reference model (like logistic regression) for interpretability and troubleshooting, because managers often need to explain why a ticket was escalated. I would also put explicit safeguards around *high* priority: the cost of predicting *high* as *low* can be severe (missed outages, SLA violations), so I would examine the confusion matrix and potentially tune thresholds or use cost-sensitive weighting to reduce "high → low" errors. If interpretability were the top priority (for auditability or policy), I would select logistic regression instead because it is easier to explain and still performs well (accuracy *0.8536*, log loss *0.4298*). Overall, the final choice depends on whether the organization values maximum triage accuracy and calibrated probabilities (favoring MLP) or simpler explanations and governance (favoring logistic regression).

13.11Learning Objectives

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to build and interpret logistic regression classifiers that produce calibrated probability estimates using sigmoid functions and log-odds <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to train and tune classification decision trees using Gini impurity or entropy as split criteria <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to evaluate classification model performance using accuracy, precision, recall, F1-score, AUC, and log loss <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to create stratified train/validation/test splits and preprocess features within scikit-learn pipelines for classification tasks <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to compare logistic regression, decision trees, k-NN, Naive Bayes, SVM, and neural network classifiers and select the appropriate model based on data characteristics and business requirements <{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to extend binary classification to multiclass problems using one-vs-rest strategies and evaluate them using per-class and macro-averaged metrics

13.12Assignment

Complete the assignment(s) below (if any):

This assessment can be taken online.